

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG ĐIỆN – ĐIỆN TỬ



ĐỒ ÁN TỐT NGHIỆP

Hệ thống Smart Building với Zigbee và BLE

Lê Đức Anh – 20224403

anh.ld224403@sis.hust.edu.vn

Chu Thiên Phú – 20224450

phu.ct224450@sis.hust.edu.vn

Ngành: Kỹ thuật Điện tử – Viễn thông

Chuyên ngành: Hệ thống nhúng thông minh và IoT

Giáo viên hướng dẫn: Phạm Thị Nhân

Chữ ký của GVHD

Hà Nội, 06/2026

LỜI CẢM ƠN

Mặc dù đã cố gắng hoàn thiện đồ án một cách tốt nhất, song do kiến thức và kinh nghiệm còn hạn chế nên không tránh khỏi những thiếu sót. Nhóm rất mong nhận được những ý kiến đóng góp quý báu từ các thầy cô để đồ án được hoàn thiện hơn.

Hà Nội, tháng 6 năm 2026

Nhóm sinh viên thực hiện

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT	4
DANH SÁCH HÌNH VẼ	6
DANH SÁCH BẢNG	7
PHÂN CHIA CÔNG VIỆC	8
1 TỔNG QUAN ĐỀ TÀI	9
1.1 Bối cảnh và lý do chọn đề tài	9
1.2 Mục tiêu của đồ án	10
1.2.1 Mục tiêu tổng quát	10
1.2.2 Mục tiêu cụ thể	10
1.3 Phạm vi và giới hạn	11
1.3.1 Phạm vi nghiên cứu và triển khai	11
1.3.2 Giới hạn của đề tài	11
1.4 Phương pháp thực hiện	11
1.5 Đóng góp chính của đồ án	12
1.6 Cấu trúc báo cáo	13
2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN	14
2.1 Khái niệm Smart Building và IoT	14
2.2 Zigbee và IEEE 802.15.4	15
2.3 Vai trò thiết bị trong mạng Zigbee	15
2.4 Topology mạng Zigbee	16
2.5 Zigbee Protocol Stack	17
2.6 Zigbee Cluster Library và mô hình thiết bị	19
2.7 Gateway trong hệ thống IoT	19
2.8 MQTT và cơ chế Publish/Subscribe	20
2.9 UART, ASH và EZSP giữa Host và EFR32 NCP	21
2.10 REST API, Cloud Backend và Database	21
2.11 Ứng dụng di động và quản lý trạng thái	22
2.12 Bảo mật trong Zigbee và IoT	22
2.13 Công nghệ sử dụng trong đồ án	23
3 PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG	24
3.1 Phạm vi và giả định thiết kế	24
3.2 Phân tích yêu cầu hệ thống	24
3.2.1 Tác nhân sử dụng hệ thống	24
3.2.2 Yêu cầu chức năng	24
3.2.3 Yêu cầu phi chức năng	25
3.3 Kiến trúc tổng thể hệ thống	25
3.4 Thiết kế thiết bị và trạng thái	26
3.4.1 Mô hình thiết bị	26

3.4.2	Trạng thái, sự kiện và lệnh	26
3.5	Luồng dữ liệu hệ thống	26
3.5.1	Luồng đi lên: thiết bị báo cáo lên Cloud	26
3.5.2	Luồng đi xuống: người dùng gửi lệnh điều khiển	26
3.5.3	Luồng tự động hóa	27
3.6	Thiết kế hợp đồng MQTT	27
3.7	Thiết kế Gateway biên	27
3.8	Thiết kế Cloud Backend, cơ sở dữ liệu và phân quyền	28
3.8.1	REST API	28
3.8.2	Cơ sở dữ liệu	28
3.8.3	Xác thực và phân quyền theo vai trò	29
3.8.4	Kiểm soát phạm vi nhà và thiết bị	29
3.9	Thiết kế Mobile App	29
3.9.1	Vai trò của Mobile App	29
3.9.2	Các nhóm màn hình chính	30
3.9.3	Luồng gọi API từ Mobile App	30
3.10	Các quyết định thiết kế và sự đánh đổi	30
4	TRIỂN KHAI HỆ THỐNG	31
4.1	Cách đọc phần triển khai	31
4.2	Môi trường triển khai và quản lý mã nguồn	31
4.3	Triển khai mạng Zigbee và thiết bị	32
4.3.1	Coordinator và NCP	32
4.3.2	Thiết bị đèn	33
4.3.3	Thiết bị công tắc	33
4.3.4	Cảm biến chuyển động	33
4.4	Triển khai Gateway	33
4.4.1	Kết nối MQTT và hàng đợi lệnh	34
4.4.2	Phân tích và thực thi lệnh	34
4.4.3	Discovery và registry thiết bị	35
4.4.4	Trạng thái, sự kiện và luật cục bộ	35
4.5	Triển khai Cloud Backend, cơ sở dữ liệu và phân quyền	35
4.5.1	Vòng đời ứng dụng và router	35
4.5.2	Mô hình dữ liệu và vòng đời lệnh	35
4.5.3	Đăng nhập và quản lý phiên	36
4.5.4	Phân quyền và phạm vi truy cập	36
4.5.5	Nhận dữ liệu MQTT	36
4.6	Triển khai Mobile App	36
4.6.1	Đăng nhập và lưu phiên	36
4.6.2	Giám sát và điều khiển thiết bị	36
4.6.3	Tự động hóa và thêm thiết bị	36
4.7	Triển khai môi trường demo	37
4.8	Luồng điều khiển đèn xuyên suốt	37
4.9	Giới hạn triển khai hiện tại	38
4.10	Tổng kết chương	38

5	KIỂM THỬ VÀ ĐÁNH GIÁ	39
5.1	Mục tiêu và nguyên tắc kiểm thử	39
5.2	Môi trường kiểm thử	39
5.3	Danh sách test case	40
5.4	Phân tích kết quả phần mềm	42
5.4.1	Cloud Backend	42
5.4.2	Mobile App	42
5.5	Evidence cần bổ sung cho kiểm thử phần cứng	42
5.6	Hạn chế và phạm vi mở rộng	42
5.7	Tổng kết chương	43
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	44
6.1	Kết luận chung	44
6.2	Kiến thức và kỹ năng đạt được	44
6.3	Hướng phát triển tiếp theo	45
6.3.1	Hoàn thiện phần cứng và kiểm thử dài hạn	45
6.3.2	Nâng cấp Gateway	45
6.3.3	Nâng cấp Cloud và Mobile App	45
6.3.4	Tăng cường bảo mật	45
6.4	Kết luận chương	46
	TÀI LIỆU THAM KHẢO	47

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Viết tắt	Ý nghĩa
ACK	Acknowledgement
AES	Advanced Encryption Standard
API	Application Programming Interface
APS	Application Support Sub-layer
BLE	Bluetooth Low Energy
CID	Correlation ID
CLI	Command Line Interface
CMD	Command
CMSIS	Cortex Microcontroller Software Interface Standard
CRLF	Carriage Return Line Feed
CSA	Connectivity Standards Alliance
CSMA/CA	Carrier Sense Multiple Access with Collision Avoidance
DMA	Direct Memory Access
EZSP	EmberZNet Serial Protocol
FFD	Full Function Device
GBL	Gecko Bootloader
GPIO	General Purpose Input/Output
GSDK	Gecko SDK
GUI	Graphical User Interface
HTTP	Hypertext Transfer Protocol
I2C	Inter-Integrated Circuit
IDE	Integrated Development Environment
IoT	Internet of Things
JSON	JavaScript Object Notation
LCD	Liquid Crystal Display
LED	Light Emitting Diode
LQI	Link Quality Indicator
LWT	Last Will and Testament (MQTT)
MAC	Media Access Control
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NCP	Network Co-Processor
NVM3	Non-Volatile Memory 3rd Generation
NWK	Network Layer
OTA	Over-the-Air
PAN	Personal Area Network
PHY	Physical Layer
PSA	Platform Security Architecture
PTI	Packet Trace Interface
QoS	Quality of Service
RAIL	Radio Abstraction Interface Layer
RBAC	Role-Based Access Control
REST	Representational State Transfer
RF	Radio Frequency
RFD	Reduced Function Device

Viết tắt	Ý nghĩa
RPi	Raspberry Pi
RTOS	Real-Time Operating System
SDK	Software Development Kit
SED	Sleepy End Device
SoC	System on Chip
SPI	Serial Peripheral Interface
SSv5	Simplicity Studio version 5
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
USB	Universal Serial Bus
VCOM	Virtual COM Port
WSTK	Wireless Starter Kit
ZC	Zigbee Coordinator
ZCL	Zigbee Cluster Library
ZDO	Zigbee Device Object
ZED	Zigbee End Device
ZR	Zigbee Router

DANH SÁCH HÌNH VẼ

1.1	Quy trình nghiên cứu và triển khai hệ thống	12
2.1	Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building	14
2.2	Minh họa topology dạng mesh trong mạng Zigbee	16
2.3	Kiến trúc tầng giao thức Zigbee	18
2.4	Mô hình broker trung tâm trong MQTT	20
2.5	Minh họa giao tiếp UART giữa hai thành phần phần cứng	21
3.1	Kiến trúc tổng thể và đường truyền dữ liệu của hệ thống	25
3.2	Luồng thiết kế cho lệnh điều khiển đèn	27
4.1	Các node Zigbee được triển khai trong mô hình thử nghiệm	32
4.2	Luồng xử lý dữ liệu trong Gateway	34
4.3	Các module Cloud Backend và luồng trao đổi với Mobile App	35
4.4	Các thành phần chính trong môi trường demo	37
4.5	Trình tự xử lý lệnh bật đèn từ Mobile App tới thiết bị	37

DANH SÁCH BẢNG

2	Phân chia công việc giữa các thành viên	8
1.1	Đóng góp chính và ý nghĩa kỹ thuật của đề án	13
2.1	Đặc điểm chính của Zigbee trong hệ thống Smart Building	15
2.2	Vai trò thiết bị trong mạng Zigbee	16
2.3	Một số cluster liên quan đến đề tài	19
2.4	Vai trò của các thành phần Cloud trong hệ thống	22
2.5	Tóm tắt công nghệ nền được sử dụng trong đề án	23
3.1	Các yêu cầu chức năng chính	25
3.2	Mô hình các loại thiết bị phiên bản hiện tại	26
3.3	Các nhóm topic MQTT chính	27
3.4	Các nhóm REST API chính	28
3.5	Các nhóm bảng dữ liệu chính	29
3.6	Ma trận quyền theo vai trò	29
3.7	Các quyết định thiết kế và sự đánh đổi	30
4.1	Bản đồ thành phần và mã nguồn triển khai	32
5.1	Môi trường dùng cho các nhóm kiểm thử	39
5.2	Test case cho thiết bị, Gateway, Cloud và Mobile App	40

PHÂN CHIA CÔNG VIỆC

Bảng 2 chia công việc thành hai vùng chính theo kiến trúc của hệ thống. Vùng Zigbee cục bộ bao gồm thiết bị, mạng Zigbee và Gateway. Vùng Cloud và lớp ứng dụng bao gồm dịch vụ phía máy chủ, cơ sở dữ liệu, phân quyền và ứng dụng di động.

Bảng 2: Phân chia công việc giữa các thành viên

STT	Công việc	Sinh viên thực hiện
Vùng 1: Zigbee cục bộ		
1	Nghiên cứu mạng Zigbee, vai trò Coordinator, Router và End Device; cấu hình bộ xử lý mạng EFR32 ở chế độ Network Co-Processor (NCP).	Lê Đức Anh
2	Xây dựng firmware cho thiết bị đèn: On/Off Server, phản hồi trạng thái và kiểm tra hoạt động của đèn.	Lê Đức Anh
3	Xây dựng firmware cho công tắc và cảm biến chuyển động: phát sinh sự kiện, báo cáo thuộc tính và kiểm tra tham gia mạng.	Chu Thiên Phú
4	Triển khai Gateway bằng ngôn ngữ C: kết nối Host–NCP, quản lý thiết bị, nhận dữ liệu Zigbee và điều phối lệnh.	Lê Đức Anh
5	Tích hợp Message Queuing Telemetry Transport (MQTT) trên Gateway: nhận lệnh, gửi trạng thái, sự kiện, thông tin thiết bị và phản hồi thực thi.	Lê Đức Anh, Chu Thiên Phú
Vùng 2: Cloud và lớp ứng dụng		
6	Thiết kế và triển khai Cloud Backend bằng FastAPI; xây dựng các Application Programming Interface (API) cho thiết bị, lệnh, sự kiện, tự động hóa và thêm thiết bị.	Chu Thiên Phú
7	Thiết kế lược đồ cơ sở dữ liệu cho người dùng, nhà, phòng, thiết bị, trạng thái, sự kiện, lệnh và luật tự động hóa.	Chu Thiên Phú
8	Triển khai đăng nhập, quản lý phiên và Role-Based Access Control (RBAC), tức cơ chế phân quyền theo vai trò và phạm vi nhà/thiết bị.	Chu Thiên Phú
9	Phát triển Mobile App bằng Flutter: đăng nhập, giám sát thiết bị, điều khiển đèn, xem sự kiện, tự động hóa và thêm thiết bị.	Lê Đức Anh
10	Tích hợp xuyên suốt Device–Gateway–Cloud–App; xây dựng test case, thu thập evidence và xử lý lỗi liên thành phần.	Lê Đức Anh, Chu Thiên Phú
11	Quản lý công việc bằng Jira; quản lý phiên bản mã nguồn bằng Git và GitHub; tổng hợp tài liệu và hoàn thiện báo cáo.	Lê Đức Anh, Chu Thiên Phú

Việc phân chia trên xác định người phụ trách chính nhưng không tách rời quá trình tích hợp. Các luồng xuyên suốt, đặc biệt là điều khiển đèn, đồng bộ trạng thái và tự động hóa, cần cả hai thành viên cùng kiểm tra vì dữ liệu đi qua nhiều lớp của hệ thống.

CHƯƠNG 1

TỔNG QUAN ĐỀ TÀI

Chương này trình bày bối cảnh và lý do lựa chọn đề tài xây dựng hệ thống IoT Zigbee Smart Building. Tiếp theo, chương xác định rõ mục tiêu tổng quát, mục tiêu cụ thể, phạm vi nghiên cứu và các giới hạn thực tế của đề án. Cuối cùng, phương pháp thực hiện, những đóng góp chính về mặt kỹ thuật và cấu trúc tổng thể của báo cáo cũng được khái quát hóa nhằm cung cấp cái nhìn toàn cảnh trước khi đi vào các nội dung kỹ thuật chi tiết.

1.1 Bối cảnh và lý do chọn đề tài

Trong những năm gần đây, sự phát triển mạnh mẽ của công nghệ mạng không dây và Internet vạn vật (Internet of Things - IoT) đã thúc đẩy việc ứng dụng tự động hóa vào quản lý vận hành tòa nhà. Smart Building (tòa nhà thông minh) không còn là khái niệm lý thuyết mà đã trở thành xu hướng tất yếu nhằm tối ưu hóa lượng điện năng tiêu thụ, nâng cao độ an toàn và mang lại trải nghiệm tiện nghi cho người sử dụng. Trong các hệ thống này, nhu cầu giám sát trạng thái môi trường, phát hiện sự hiện diện của con người và điều khiển các thiết bị phụ tải (như đèn chiếu sáng, công tắc) là những bài toán cốt lõi cần giải quyết.

Đối với mạng kết nối nội bộ (local network) trong tòa nhà, việc sử dụng các công nghệ truyền thông không dây phổ biến như Wi-Fi thường bộc lộ những hạn chế lớn về mặt tiêu thụ năng lượng và khả năng chịu tải của điểm truy cập trung tâm khi số lượng thiết bị lên tới hàng trăm node. Do đó, các công thức truyền thông tiết kiệm năng lượng, hỗ trợ cấu trúc liên kết mạng lưới (mesh network) như Zigbee được xem là giải pháp tối ưu. Zigbee hoạt động trên tiêu chuẩn IEEE 802.15.4, cho phép các thiết bị tự động định tuyến gói tin qua nhau, giúp mở rộng phạm vi phủ sóng mà không yêu cầu công suất phát lớn, đồng thời giảm thiểu đáng kể chi phí duy trì nguồn pin cho các cảm biến.

Tuy nhiên, các mạng Zigbee cục bộ không thể tự kết nối trực tiếp với các ứng dụng di động hoặc hệ thống quản lý từ xa qua mạng IP diện rộng. Điều này đòi hỏi sự xuất hiện của một thiết bị trung gian gọi là Gateway. Thiết bị Gateway đóng vai trò làm cầu nối chuyển đổi giữa giao thức Zigbee RF tầm ngắn và mạng IP (chạy các giao thức như MQTT, HTTP) kết nối lên Cloud. Sự phối hợp giữa Gateway và hệ thống đám mây (Cloud Backend) cho phép lưu trữ lịch sử vận hành, xử lý các kịch bản tự động hóa phức tạp và cung cấp các giao diện lập trình ứng dụng (API) cho ứng dụng di động của người dùng cuối.

Hơn nữa, các giải pháp smart building thương mại hiện nay thường là các hệ sinh thái đóng (closed ecosystem) của các nhà sản xuất lớn, gây khó khăn cho việc tích hợp thiết bị của bên thứ ba và hạn chế khả năng tùy biến logic điều khiển. Xuất phát từ nhu cầu nghiên cứu một nền tảng quản lý thiết bị mở, có khả năng làm chủ từ firmware thiết bị, logic xử lý tại gateway cho đến kiến trúc cloud backend, đề tài "**Hệ thống IoT Zigbee Smart Building**" đã được lựa chọn để thực hiện. Đề án tập trung nghiên cứu thiết kế và tích hợp đồng bộ các lớp thiết bị, gateway và dịch vụ đám mây nhằm đảm bảo luồng dữ liệu hai chiều nhanh chóng, tin cậy và có khả năng mở rộng.

1.2 Mục tiêu của đề án

Để giải quyết các bài toán đặt ra trong bối cảnh trên, đề án xác định rõ mục tiêu tổng quát và các mục tiêu cụ thể cần đạt được.

1.2.1 Mục tiêu tổng quát

Mục tiêu tổng quát của đề án là thiết kế, chế tạo và tích hợp hoàn chỉnh một hệ thống IoT quản lý Smart Building dựa trên chuẩn truyền thông không dây Zigbee, kết nối qua Gateway C tới hệ thống Cloud Backend, hỗ trợ giám sát trạng thái thiết bị thực tế gần thời gian thực và tự động hóa điều khiển phụ tải.

1.2.2 Mục tiêu cụ thể

Để hiện thực hóa mục tiêu tổng quát, đề án đề ra các mục tiêu cụ thể cho từng thành phần trong hệ thống:

Phía thiết bị cuối (End Devices): Thiết kế và nạp firmware cho các thiết bị Zigbee 3.0 dựa trên vi điều khiển EFR32MG12 bao gồm thiết bị đèn (Z3Light), công tắc (Z3Switch) và cảm biến hiện diện (Occupancy/Motion Sensor) hoạt động ổn định trong mạng mesh Zigbee.

Phía Gateway trung gian: Triển khai ứng dụng Z3Gateway dạng tiến trình đơn (single-process) viết bằng ngôn ngữ C chạy trên Linux host. Gateway tích hợp MQTT client để nhận command từ Cloud, chuyển thành thao tác Zigbee và publish trạng thái, sự kiện hoặc phản hồi trở lại broker.

Phía dịch vụ đám mây (Cloud Backend): Xây dựng Cloud Backend sử dụng framework FastAPI (Python 3.12) kết nối cơ sở dữ liệu PostgreSQL. Hệ thống đám mây phải tiếp nhận các bản tin telemetry báo cáo trạng thái, quản lý vòng đời lệnh điều khiển, lưu trữ lịch sử sự kiện và cung cấp hệ thống REST API chuẩn hóa.

Phía ứng dụng di động (Mobile App): Xây dựng ứng dụng Flutter hỗ trợ đăng nhập, xem danh sách và trạng thái thiết bị, gửi lệnh bật/tắt đèn, xem sự kiện, quản lý luật tự động hóa và thực hiện luồng thêm thiết bị theo quyền được cấp.

Về mặt kiểm thử và đánh giá: Xây dựng bộ test tự động cho Cloud và Mobile App, đồng thời định nghĩa các test case xuyên suốt cho thiết bị, Gateway, MQTT, Cloud và App. Kết quả chỉ được kết luận khi có output, log, API response, ảnh hoặc video tương ứng.

1.3 Phạm vi và giới hạn

Việc xác định phạm vi nghiên cứu và giới hạn thực thi giúp đồ án tập trung vào các vấn đề kỹ thuật trọng tâm, tránh phân tán nguồn lực vào các tính năng thương mại phức tạp.

1.3.1 Phạm vi nghiên cứu và triển khai

Đồ án tập trung thực hiện các nội dung sau:

Thiết lập mạng Zigbee cục bộ: Sử dụng một Coordinator chạy firmware NCP (Network Co-Processor) kết nối qua UART/ASH với Gateway host.

Triển khai kịch bản gia nhập mạng: Thực hiện permit-join, đồng bộ trạng thái thực tế lên cloud (reported state) và truyền đạt lệnh trạng thái mong muốn (desired state) qua giao thức MQTT.

Đồng bộ trạng thái thiết bị và kịch bản tự động hóa cục bộ: Hỗ trợ local automation cơ bản thông qua cơ chế APS Binding trực tiếp giữa công tắc và đèn trong mạng Zigbee mesh.

Theo dõi vòng đời của lệnh điều khiển: Theo dõi chặt chẽ quá trình xử lý lệnh từ Cloud xuống thiết bị qua các pha trạng thái rõ ràng (accepted → queued → sent → executed/failed/-timeout).

Triển khai hệ thống Cloud Backend: Deploy hệ thống trên hạ tầng điện toán đám mây dưới dạng các container Docker được đóng gói và cấu hình hoàn chỉnh.

1.3.2 Giới hạn của đề tài

Do hạn chế về mặt thời gian và trang thiết bị phần cứng, đồ án tồn tại một số giới hạn như sau:

Quy mô thử nghiệm: Do hạn chế về mặt thời gian và trang thiết bị phần cứng, hệ thống chỉ dừng lại ở quy mô thử nghiệm phòng lab (lab-scale) với số lượng thiết bị demo hạn chế (khoảng 3-5 node), chưa được triển khai thực tế trên quy mô tòa nhà nhiều tầng với mật độ cản trở vật lý cao.

Phạm vi thiết bị và Gateway: Phiên bản hiện tại quản lý một Gateway và ba loại thiết bị chính là light, switch và motion. Hệ thống chưa kiểm thử nhiều Gateway, nhiều site hoặc các node chấp hành khác như ổ cắm, relay, dimmer, rèm và thiết bị điều hòa.

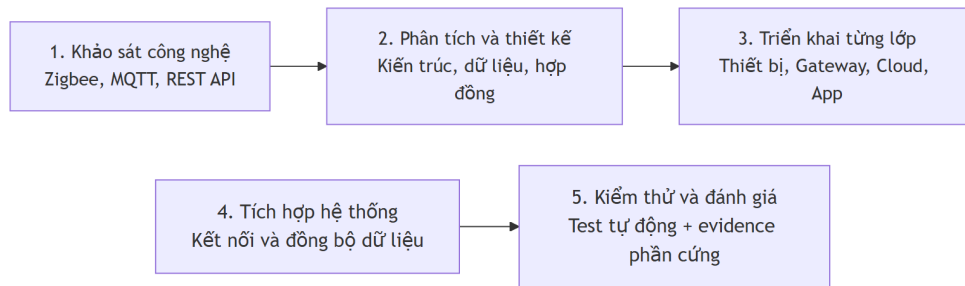
Ứng dụng di động: Ứng dụng Flutter đã có repository gọi API thật, đăng nhập, quản lý phiên, điều khiển đèn, tự động hóa và thêm thiết bị. Tuy nhiên, báo cáo chưa có đủ evidence để kết luận toàn bộ luồng đã ổn định trên nhiều loại điện thoại và trong thời gian vận hành dài.

Cập nhật trạng thái: Mobile App chủ yếu gọi REST API và truy vấn lại trạng thái lệnh. Hệ thống chưa dùng kênh đẩy dữ liệu như WebSocket hoặc Server-Sent Events, vì vậy độ trễ hiển thị phụ thuộc chu kỳ gọi API và trạng thái kết nối.

Vận hành và kiểm thử: Phiên bản hiện tại còn thiếu evidence phần cứng cho một số test xuyên suốt, chưa kiểm thử tải lớn và chưa hoàn thiện bộ tài liệu vận hành production mà contract test yêu cầu. Vì vậy, báo cáo chỉ khẳng định các kết quả đã có log hoặc output kiểm thử tương ứng.

1.4 Phương pháp thực hiện

Để thực hiện đồ án một cách khoa học, quy trình nghiên cứu và phát triển được chia làm 5 bước tuần tự như được mô tả trong Hình 1.1.



Hình 1.1: Quy trình nghiên cứu và triển khai hệ thống

Quy trình thực hiện bao gồm các giai đoạn cụ thể sau:

Giai đoạn 1 – Khảo sát và nghiên cứu công nghệ: Nghiên cứu tài liệu kỹ thuật về chuẩn Zigbee 3.0, thư viện ZCL (Zigbee Cluster Library) của Silicon Labs, giao thức truyền thông MQTT và kiến trúc API hướng tài nguyên sử dụng REST.

Giai đoạn 2 – Thiết kế hệ thống: Xác định sơ đồ khối kiến trúc hệ thống 4 lớp, thiết kế cấu trúc cơ sở dữ liệu PostgreSQL, xây dựng hợp đồng trao đổi bản tin MQTT (MQTT Contract JSON Schema) và cấu hình các topic phân cấp.

Giai đoạn 3 – Triển khai: Lập trình firmware cho EFR32MG12; viết Z3Gateway bằng ngôn ngữ C; xây dựng Cloud API bằng FastAPI.

Giai đoạn 4 – Tích hợp hệ thống: Kết nối Gateway host với chip NCP qua UART, kết nối Gateway lên Mosquitto Broker chạy trên container Docker, thiết lập liên kết đồng bộ giữa cơ sở dữ liệu và MQTT Client chạy nền trên Cloud.

Giai đoạn 5 – Kiểm thử và đánh giá: Chạy các kịch bản kiểm thử tích hợp tự động bằng pytest cho backend và thực thi kiểm thử thủ công end-to-end trực tiếp trên phần cứng thật để thu thập log vận hành.

1.5 Đóng góp chính của đề án

Kết quả thực hiện đề án mang lại một số đóng góp về kiến trúc tích hợp, quản lý lệnh, nhận dạng thiết bị và phương pháp kiểm thử. Bảng 1.1 tóm tắt các đóng góp và ý nghĩa kỹ thuật tương ứng.

Bảng 1.1: Đóng góp chính và ý nghĩa kỹ thuật của đồ án

Giải pháp phát triển	Ý nghĩa thực tế và kỹ thuật
Kiến trúc Gateway C đơn tiến trình tích hợp MQTT client	Giữ đường xử lý command, report và event trong cùng ứng dụng Gateway, làm rõ trách nhiệm giữa MQTT callback, hàng đợi lệnh và Zigbee callback.
Mô hình theo dõi vòng đời lệnh điều khiển (Command Lifecycle)	Cho phép theo dõi tường minh trạng thái xử lý gói tin qua các pha (accepted, queued, sent, executed, failed, timeout), giải quyết bài toán lệnh bị treo hoặc mất gói vô tuyến mà cloud không phát hiện được.
Chuẩn hóa mô hình nhận dạng thiết bị qua device_id logic	Tách biệt định danh cơ sở dữ liệu khỏi địa chỉ phần cứng cố định (EUI-64) hoặc địa chỉ động (16-bit nwk_addr), tăng tính bảo mật và cho phép linh hoạt thay thế phần cứng lỗi.
Khung test case xuyên suốt và nguyên tắc evidence	Tách rõ kết quả test phần mềm đã có output với test phần cứng còn cần log, API response, ảnh hoặc video trước khi đánh giá.

Chương 5 trình bày output của các nhóm test phần mềm và chỉ rõ những test phần cứng còn thiếu evidence. Cách ghi nhận này giúp phân biệt phần đã kiểm chứng tự động với phần cần tiếp tục đo trên mô hình thực.

1.6 Cấu trúc báo cáo

Nội dung báo cáo đồ án tốt nghiệp được tổ chức thành 6 chương chính sau đây:

Chương 1 – Tổng quan đề tài: Giới thiệu bối cảnh, lý do chọn đề tài, xác định mục tiêu, phạm vi nghiên cứu, phương pháp thực hiện và cấu trúc tổng thể của báo cáo.

Chương 2 – Cơ sở lý thuyết và công nghệ nền: Trình bày nền tảng lý thuyết về giao thức mạng Zigbee, thư viện cluster ZCL, giao thức MQTT, giao tiếp nối tiếp UART/ASH và các kit vi điều khiển phần cứng được sử dụng.

Chương 3 – Phân tích yêu cầu và thiết kế hệ thống: Phân tích các yêu cầu chức năng, phi chức năng, xây dựng sơ đồ kiến trúc tổng thể, mô hình dữ liệu, đặc tả giao diện API và hợp đồng trao đổi bản tin MQTT Contract.

Chương 4 – Triển khai hệ thống: Trình bày chi tiết quá trình lập trình firmware thiết bị, viết ứng dụng Gateway C, lập trình Cloud Backend bằng FastAPI và giải quyết các lỗi thực tế khi deploy lên EC2.

Chương 5 – Kiểm thử và đánh giá: Thiết lập kế hoạch kiểm thử, trình bày kết quả chạy các test case cụ thể, ghi nhận các bằng chứng logs/traces và phân tích ưu điểm cũng như hạn chế của hệ thống.

Chương 6 – Kết luận và hướng phát triển: Tổng kết các kết quả có evidence, nêu kiến thức tích lũy và đề xuất các hướng mở rộng về độ ổn định, giám sát, bảo mật và cập nhật trạng thái gần thời gian thực.

CHƯƠNG 2

CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ NỀN

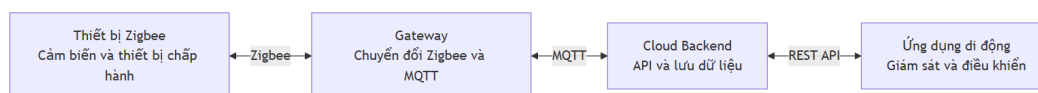
Chương 2 trình bày cơ sở lý thuyết và công nghệ dùng để xây dựng hệ thống Internet of Things (IoT) cho Smart Building. Nội dung gồm khái niệm tòa nhà thông minh, mạng Zigbee, mô hình thiết bị theo Zigbee Cluster Library, vai trò của Gateway, giao thức MQTT, giao tiếp giữa Host và EFR32 NCP, Cloud Backend, ứng dụng di động và các nguyên tắc bảo mật cơ bản. Chương chỉ trình bày kiến thức nền, không đi vào mã nguồn hoặc kết quả thực nghiệm.

2.1 Khái niệm Smart Building và IoT

Smart Building, hay tòa nhà thông minh, là công trình sử dụng cảm biến, thiết bị chấp hành, mạng truyền thông và phần mềm để giám sát hoặc điều khiển các chức năng trong tòa nhà. Mục tiêu không chỉ là điều khiển từ xa mà còn là tạo vòng phản hồi: thu thập trạng thái, xử lý dữ liệu, thực hiện hành động và ghi nhận kết quả. Trong đề tài này, phạm vi Smart Building được thu nhỏ thành mô hình một phòng thông minh có đèn, công tắc, cảm biến chuyển động, Gateway, Cloud Backend và ứng dụng di động.

Internet of Things (IoT), hay Internet vạn vật, là mô hình kết nối thiết bị vật lý với phần mềm thông qua mạng truyền thông. Thiết bị trong hệ thống IoT thường có cảm biến, bộ xử lý, phần mềm nhúng và khả năng trao đổi dữ liệu. Cảm biến thu thập trạng thái, thiết bị chấp hành thực hiện hành động vật lý, Gateway nối mạng thiết bị cục bộ với hệ thống phía trên, còn Cloud Backend lưu dữ liệu và cung cấp Application Programming Interface (API) cho ứng dụng.

Một luồng IoT cơ bản thường bắt đầu từ việc thu thập dữ liệu, truyền dữ liệu, xử lý dữ liệu, phát sinh lệnh điều khiển và hiển thị lại trạng thái cho người dùng. Trong đề án này, người dùng thao tác trên Flutter Mobile App; App gọi FastAPI Cloud REST API; Cloud trao đổi trạng thái và lệnh qua Mosquitto MQTT Broker; Z3Gateway C nhận bản tin MQTT và chuyển thành Zigbee command qua EFR32 NCP; thiết bị Zigbee phản hồi trạng thái theo chiều ngược lại.



Hình 2.1: Kiến trúc tổng quát của hệ thống IoT Zigbee Smart Building

Cách tổ chức trên giúp tách rõ trách nhiệm giữa các lớp. Mạng Zigbee xử lý truyền thông cục bộ cho thiết bị công suất thấp. Gateway đảm nhiệm chuyển đổi giữa Zigbee và MQTT. Cloud lưu dữ liệu và cung cấp API. App tập trung vào giao diện giám sát và điều khiển. Khi thiết kế hệ thống Smart Building, nhóm thực hiện cần cân bằng giữa xử lý cục bộ và xử lý trên Cloud. Xử lý cục bộ giúp giảm độ trễ trong một số tình huống, còn Cloud thuận lợi hơn cho lưu trữ, quản lý từ xa và mở rộng dữ liệu.

2.2 Zigbee và IEEE 802.15.4

Zigbee là giao thức truyền thông không dây công suất thấp, thường được dùng trong các hệ thống cảm biến và điều khiển [1]. Đặc điểm của Zigbee là tốc độ dữ liệu không cao nhưng tiêu thụ năng lượng thấp, hỗ trợ nhiều thiết bị và có khả năng tổ chức mạng linh hoạt. Các đặc điểm này phù hợp với Smart Building, vì các thiết bị như đèn, công tắc và cảm biến hiện diện chủ yếu trao đổi trạng thái hoặc lệnh ngắn, không truyền dữ liệu dung lượng lớn.

IEEE 802.15.4 là chuẩn ở tầng vật lý và tầng Medium Access Control (MAC) cho mạng cá nhân không dây công suất thấp. Zigbee sử dụng nền tảng này và bổ sung thêm các tầng phía trên như network layer, security và application layer [3]. Có thể hiểu IEEE 802.15.4 quy định cách truyền nhận khung dữ liệu ở mức radio, còn Zigbee quy định thêm cách thiết bị tạo mạng, tham gia mạng, định tuyến và biểu diễn chức năng ứng dụng.

Trong một hệ thống IoT, Zigbee không thay thế hoàn toàn Wi-Fi hoặc Ethernet. Zigbee phù hợp hơn ở lớp thiết bị cục bộ, nơi yêu cầu tiết kiệm năng lượng và truyền các bản tin nhỏ. Đối lại, thiết bị Zigbee thường không giao tiếp trực tiếp với Internet, nên hệ thống cần Gateway làm cầu nối lên Cloud hoặc App.

Bảng 2.1: Đặc điểm chính của Zigbee trong hệ thống Smart Building

Đặc điểm	Ý nghĩa trong hệ thống
Công suất thấp	Phù hợp với cảm biến, công tắc và thiết bị nhúng cần hoạt động lâu dài.
Băng thông thấp	Không phù hợp truyền dữ liệu lớn, nhưng đủ cho trạng thái, sự kiện và lệnh điều khiển.
Hỗ trợ mesh	Có thể mở rộng vùng phủ thông qua các thiết bị có khả năng định tuyến.
Mô hình ZCL	Chuẩn hóa chức năng thiết bị thành cluster, attribute và command.
Cần Gateway	Thiết bị Zigbee không giao tiếp trực tiếp với App/Cloud qua Internet, nên cần Gateway làm cầu nối.

Trong đồ án, Zigbee được dùng cho lớp thiết bị cục bộ. EFR32 NCP đóng vai trò radio coordinator, còn các thiết bị như Light, Switch và Occupancy Sensor là các thiết bị Zigbee trong mạng. Nhờ đó, phần thiết bị nhúng được tách khỏi phần Cloud/App, đồng thời vẫn có thể đồng bộ trạng thái và nhận lệnh điều khiển thông qua Gateway.

2.3 Vai trò thiết bị trong mạng Zigbee

Mạng Zigbee phân chia thiết bị theo vai trò để mạng có thể hình thành, duy trì kết nối và truyền dữ liệu. Ba vai trò cơ bản gồm Coordinator, Router và End Device. Coordinator tạo và quản lý mạng. Router có thể chuyển tiếp gói tin để mở rộng vùng phủ. End Device là thiết bị cuối, thực hiện chức năng ứng dụng như đo trạng thái hoặc nhận lệnh điều khiển.

Coordinator là thiết bị khởi tạo mạng Zigbee. Thiết bị này chọn kênh, thiết lập Personal Area Network (PAN), quản lý quá trình cho phép thiết bị khác tham gia mạng và thường liên quan đến Trust Center trong cơ chế bảo mật. Trong kiến trúc Host-NCP, vai trò coordinator radio có thể đặt trên EFR32 NCP, còn ứng dụng Gateway chạy ở phía Host.

Router là thiết bị có khả năng chuyển tiếp gói tin cho các node khác. Trong môi trường tòa nhà, Router giúp mở rộng vùng phủ khi thiết bị không thể liên lạc trực tiếp với Coordinator.

Tuy nhiên, Router thường cần nguồn cấp ổn định hơn so với End Device vì phải duy trì chức năng định tuyến.

End Device là thiết bị cuối trong mạng. Một End Device có thể là cảm biến hiện diện, công tắc hoặc đèn. Thiết bị này tập trung vào chức năng ứng dụng, ví dụ gửi trạng thái có người/không có người hoặc nhận lệnh bật/tắt. Với các thiết bị dùng pin, End Device có thể được tối ưu để tiết kiệm năng lượng bằng cách ngủ trong một số khoảng thời gian.

Bảng 2.2: Vai trò thiết bị trong mạng Zigbee

Vai trò	Nhiệm vụ chính	Ví dụ trong đề tài
Coordinator	Tạo mạng, quản lý mạng và hỗ trợ thiết bị tham gia mạng.	EFR32 NCP gắn với Gateway.
Router	Chuyển tiếp gói tin, mở rộng vùng phủ mạng.	Thiết bị luôn cấp nguồn nếu được cấu hình hỗ trợ routing.
End Device	Thực hiện chức năng ứng dụng, gửi trạng thái hoặc nhận lệnh.	Light, Switch, Occupancy Sensor.

Phần này không đi sâu vào thuật toán định tuyến của Zigbee. Mục tiêu là làm rõ vì sao Gateway cần một coordinator radio để quản lý mạng, và vì sao các thiết bị như light, switch, sensor có thể được tổ chức như các node trong cùng một mạng Zigbee.

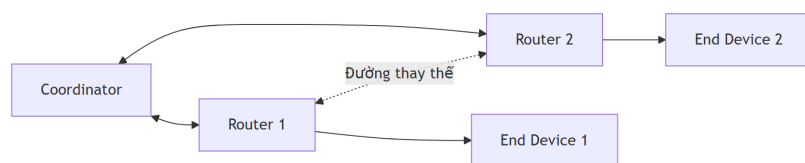
2.4 Topology mạng Zigbee

Topology mô tả cách các node trong mạng được tổ chức và liên kết với nhau. Trong Zigbee, ba dạng topology thường gặp là star, tree và mesh. Việc lựa chọn topology ảnh hưởng trực tiếp đến vùng phủ, độ phức tạp khi quản lý mạng và khả năng mở rộng của hệ thống.

Với star topology, các thiết bị trao đổi dữ liệu thông qua một node trung tâm. Mô hình này đơn giản, dễ kiểm soát và phù hợp với phạm vi nhỏ. Hạn chế của nó là vùng phủ phụ thuộc nhiều vào node trung tâm; nếu thiết bị ở xa hoặc bị vật cản, chất lượng kết nối có thể giảm.

Tree topology tổ chức mạng theo dạng phân cấp. Cách tổ chức này phù hợp khi muốn quản lý theo nhánh, ví dụ theo tầng hoặc theo khu vực trong tòa nhà. Hạn chế là các node phía dưới phụ thuộc vào node trung gian trong nhánh; khi node trung gian gặp sự cố, các node phía sau có thể bị ảnh hưởng.

Mesh topology cho phép một số node chuyển tiếp gói tin qua nhiều đường khác nhau. Đây là topology đáng chú ý trong Smart Building vì môi trường tòa nhà thường có tường, vật cản và khoảng cách giữa các thiết bị. Mesh giúp mở rộng vùng phủ tốt hơn star topology, nhưng việc quản lý mạng và debug cũng phức tạp hơn.



Hình 2.2: Minh họa topology dạng mesh trong mạng Zigbee

Trong đề án, Zigbee được lựa chọn để tạo mạng cục bộ cho các thiết bị trong phòng/lab. Dù số lượng thiết bị ở giai đoạn đề án chưa lớn, nền tảng mesh của Zigbee vẫn có ý nghĩa vì nó

cho phép hệ thống mở rộng sang nhiều node hơn khi cần.

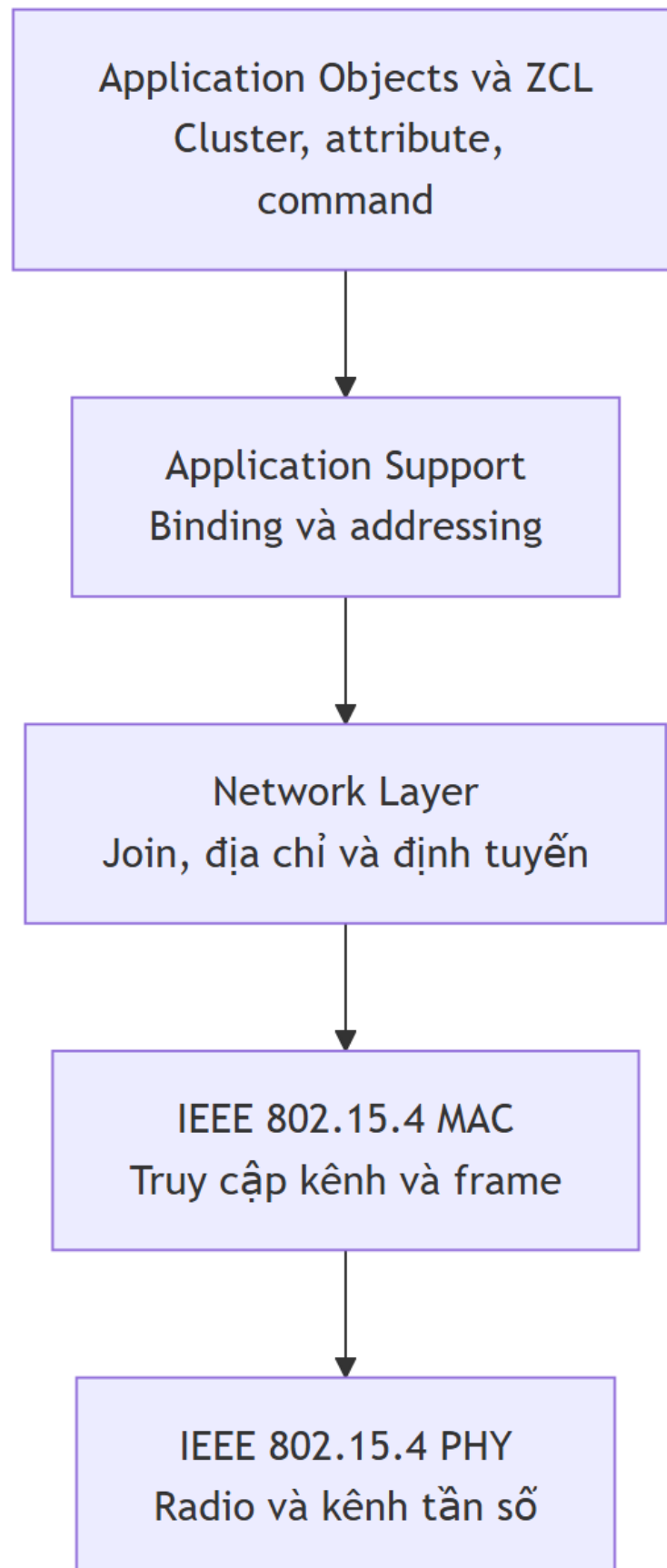
2.5 Zigbee Protocol Stack

Protocol Stack là tập hợp các tầng giao thức cùng tham gia vào quá trình truyền thông. Mỗi tầng đảm nhiệm một nhóm chức năng riêng, từ truyền tín hiệu vô tuyến ở tầng thấp đến biểu diễn lệnh ứng dụng ở tầng cao. Cách phân tầng này giúp hệ thống mạng dễ chuẩn hóa hơn, đồng thời giúp người phát triển xác định lỗi đang nằm ở tầng truyền dẫn, tầng mạng hay tầng ứng dụng.

Tầng Physical (PHY) và MAC của Zigbee dựa trên IEEE 802.15.4. Tầng PHY liên quan đến tần số, kênh truyền và việc phát/thu tín hiệu vô tuyến. Tầng MAC quản lý cách các node truy cập môi trường truyền, định dạng khung dữ liệu cơ bản và một số cơ chế trao đổi ở mức liên kết.

Tầng Network xử lý địa chỉ mạng, quá trình hình thành mạng, quá trình tham gia mạng và định tuyến. Khi một thiết bị mới join mạng, tầng này hỗ trợ cấp địa chỉ và xác định cách thiết bị gửi dữ liệu đến node đích. Với topology mesh, tầng Network có vai trò đáng kể vì gói tin có thể đi qua node trung gian.

Ở phía trên, Application Support (APS) và Application/ZCL xử lý truyền thông ở mức ứng dụng. APS hỗ trợ addressing và binding, còn ZCL định nghĩa cluster, attribute và command. Nhờ đó, lệnh bật/tắt đèn trong hệ thống Zigbee được biểu diễn theo mô hình chuẩn, thay vì dùng chuỗi dữ liệu tự do do từng nhóm tự đặt.



Hình 2.3: Kiến trúc tầng giao thức Zigbee

Ưu điểm của Protocol Stack chuẩn là tính tương thích và khả năng mở rộng tốt hơn. Hạn chế là người phát triển phải nắm thêm nhiều tầng giao thức khi debug. Với đề án này, lợi ích của việc dùng stack chuẩn lớn hơn độ phức tạp phát sinh, vì hệ thống cần làm việc với nhiều loại thiết bị và cần ánh xạ rõ chức năng của từng thiết bị.

2.6 Zigbee Cluster Library và mô hình thiết bị

Zigbee Cluster Library (ZCL) là bộ đặc tả chuẩn hóa cách mô tả chức năng của thiết bị Zigbee [2]. Nếu các tầng thấp của Zigbee quy định cách dữ liệu đi qua mạng, ZCL quy định dữ liệu đó mang ý nghĩa gì ở tầng ứng dụng. Đây là phần cần thiết trong đề án vì hệ thống sử dụng các thiết bị như Light, Switch và Occupancy Sensor.

Trong ZCL, cluster là nhóm chức năng của thiết bị. Attribute là trạng thái hoặc thuộc tính nằm trong cluster. Command là lệnh được gửi đến thiết bị hoặc sự kiện do thiết bị phát ra. Ba khái niệm này tạo ra cách biểu diễn thống nhất giữa Gateway và thiết bị Zigbee. Ví dụ, lệnh bật/tắt đèn có thể được biểu diễn bằng On/Off Cluster thay vì một chuỗi text tự định nghĩa.

Bảng 2.3: Một số cluster liên quan đến đề tài

Cluster	Identifier	Vai trò
On/Off Cluster	0x0006	Biểu diễn chức năng bật/tắt, phù hợp với light hoặc switch.
Level Control Cluster	0x0008	Biểu diễn mức độ, ví dụ độ sáng của đèn nếu thiết bị hỗ trợ.
Occupancy Sensing Cluster	0x0406	Biểu diễn trạng thái phát hiện hiện diện, phù hợp với occupancy sensor.

Trong luồng điều khiển, App tạo yêu cầu ở mức người dùng, Cloud chuyển yêu cầu thành command logic, Gateway ánh xạ command sang action Zigbee, Zigbee stack đóng gói action theo cluster/command và thiết bị thực hiện lệnh. Ở chiều ngược lại, thiết bị gửi trạng thái mới, Gateway nhận báo cáo và publish reported state để Cloud lưu trữ và App hiển thị.

Việc sử dụng ZCL khiến phần phát triển phức tạp hơn so với tự đặt một payload đơn giản. Tuy nhiên, ZCL giúp hệ thống tránh tình trạng mỗi loại thiết bị dùng một cách biểu diễn riêng. Với đề án này, lợi ích đó thể hiện rõ ở việc Gateway có thể xử lý lệnh theo loại thiết bị và chức năng, ví dụ tách lệnh bật/tắt light khỏi trạng thái phát hiện hiện diện của sensor.

2.7 Gateway trong hệ thống IoT

Gateway là thành phần nối mạng thiết bị cục bộ với hệ thống phần mềm phía trên. Trong hệ thống IoT Zigbee, thiết bị Zigbee không trực tiếp giao tiếp với REST API hoặc App. Gateway vì vậy cần chuyển đổi giữa hai miền: một bên là mạng Zigbee với cluster, attribute và command; bên còn lại là Cloud, MQTT và các dữ liệu ứng dụng.

Trong đề án này, Gateway được hiểu theo kiến trúc hiện tại: Z3Gateway C Host App kết nối với EFR32 NCP và tích hợp MQTT trực tiếp thông qua libmosquitto. Cloud và Mobile App trao đổi dữ liệu qua MQTT Broker; Z3Gateway C nhận command từ MQTT, gọi Zigbee action qua Ember AF/EZSP, sau đó publish reported state hoặc event trở lại MQTT khi thiết bị phản hồi.

Gateway không chỉ chuyển tiếp dữ liệu thô. Thành phần này còn parse payload, xác định

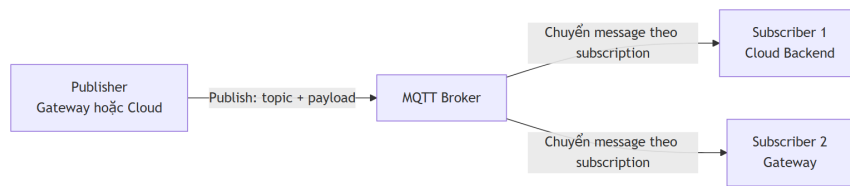
loại thiết bị, ánh xạ command theo device_type, gọi API của Zigbee stack, chuẩn hóa telemetry và publish trạng thái. Trong một số trường hợp, Gateway cũng có thể xử lý local automation ở mức cơ bản, ví dụ sự kiện từ switch hoặc sensor dẫn đến lệnh điều khiển light.

Khi đặt nhiều logic ở Gateway, hệ thống có thể phản hồi nhanh hơn và giảm phụ thuộc vào Cloud trong một số tình huống. Tuy nhiên, Gateway cũng trở thành thành phần khó bảo trì hơn nếu trách nhiệm bị mở rộng quá nhiều. Vì vậy, ranh giới hợp lý là Gateway xử lý kết nối thiết bị và các phản ứng cục bộ cần thiết, còn Cloud đảm nhiệm lưu trữ, API, lịch sử và quản lý người dùng.

2.8 MQTT và cơ chế Publish/Subscribe

Message Queuing Telemetry Transport (MQTT) là giao thức messaging nhẹ, được sử dụng phổ biến trong IoT để truyền telemetry và command giữa các thành phần phân tán [4, 5]. MQTT hoạt động theo mô hình publish/subscribe. Client không gửi dữ liệu trực tiếp cho nhau, mà publish message vào topic; broker nhận message và chuyển tiếp cho các client đang subscribe topic đó.

Các thuật ngữ cơ bản của MQTT gồm broker, client, topic, payload, publish, subscribe, Quality of Service (QoS), retained message và Last Will and Testament (LWT). Broker là trung tâm điều phối message. Client là thành phần kết nối tới broker, chẳng hạn Cloud Backend hoặc Gateway. Topic là đường dẫn logic dùng để phân loại message. Payload là nội dung message. Publish là hành động gửi message, còn subscribe là đăng ký nhận message.



Hình 2.4: Mô hình broker trung tâm trong MQTT

Trong hệ thống của đề án, Cloud và Gateway cùng kết nối tới Mosquitto MQTT Broker. Ở chiều điều khiển, App gọi REST API lên Cloud, Cloud publish command request xuống topic mà Gateway subscribe. Gateway nhận message, parse payload và chuyển thành Zigbee action. Ở chiều trạng thái, Gateway nhận reported state hoặc event từ mạng Zigbee, sau đó publish lên MQTT để Cloud lưu vào database và App hiển thị lại cho người dùng.

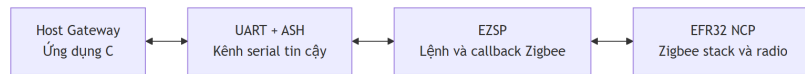
QoS quy định mức đảm bảo truyền message. QoS 0 gửi tối đa một lần, nhẹ nhất nhưng có thể mất message. QoS 1 đảm bảo message đến ít nhất một lần, nhưng bên nhận cần xử lý khả năng trùng lặp. QoS 2 đảm bảo đúng một lần ở mức giao thức, nhưng phức tạp và tốn tài nguyên hơn. Retained message cho phép broker giữ message cuối cùng của một topic, nhờ đó subscriber mới có thể nhận trạng thái gần nhất. LWT giúp broker phát message thay mặt client nếu client mất kết nối bất thường.

MQTT phù hợp với đề án vì bản tin trạng thái và lệnh điều khiển thường nhỏ, gửi theo sự kiện và cần tách rời Cloud với Gateway. Hạn chế chính là hệ thống phải thiết kế topic và payload rõ ràng. Nếu contract MQTT không ổn định, việc debug và mở rộng hệ thống sẽ khó khăn hơn. Vì vậy, MQTT được xem là contract chính giữa Cloud và Gateway, còn chi tiết topic/payload cụ thể được trình bày ở phần thiết kế và triển khai.

2.9 UART, ASH và EZSP giữa Host và EFR32 NCP

Universal Asynchronous Receiver/Transmitter (UART) là chuẩn giao tiếp nối tiếp bất đồng bộ giữa hai thiết bị [6]. UART thường sử dụng các đường TX, RX và GND; hai bên truyền thông phải thống nhất baud rate, số bit dữ liệu, stop bit và parity. Trong đồ án, UART được xét chủ yếu như đường truyền vật lý/serial giữa Host Gateway và EFR32 NCP, không phải giao thức ứng dụng giữa Gateway và Cloud.

Network Co-Processor (NCP) là mô hình trong đó chip radio xử lý các phần Zigbee network/stack ở mức thấp hơn, còn Host chạy ứng dụng Gateway ở mức cao hơn. Host và NCP cần một cơ chế trao đổi lệnh, sự kiện và dữ liệu. Với nền tảng Silicon Labs, EmberZNet Serial Protocol (EZSP) và Asynchronous Serial Host (ASH) được dùng để hỗ trợ giao tiếp giữa Host và NCP [7, 8].



Hình 2.5: Minh họa giao tiếp UART giữa hai thành phần phần cứng

Điểm cần phân biệt là UART/EZSP/ASH thuộc biên giao tiếp giữa Host và NCP, còn application contract của hệ thống nằm ở MQTT và Zigbee/ZCL action mapping. Do đó, các frame UART tự định nghĩa trong tài liệu cũ không nên được xem là contract production hiện tại của đồ án.

Mô hình Host–NCP giúp nhóm thực hiện tận dụng stack Zigbee và phần radio của Silicon Labs, thay vì phải tự xử lý toàn bộ logic mạng ở mức thấp. Hạn chế là quá trình debug cần xem xét thêm lớp serial, EZSP/ASH và tương tác giữa Host với NCP. Phần này được trình bày ở mức vừa đủ để giải thích vì sao Gateway C có thể điều khiển mạng Zigbee thông qua EFR32 NCP.

2.10 REST API, Cloud Backend và Database

Cloud Backend là lớp phần mềm trung tâm ở phía server. Trong hệ thống Smart Building, Cloud Backend cung cấp REST API cho App, nhận trạng thái từ Gateway, lưu dữ liệu vào database và gửi command xuống Gateway khi người dùng hoặc rule automation yêu cầu. Nhờ có Backend, App không cần làm việc trực tiếp với Zigbee, cluster hoặc NCP.

Representational State Transfer (REST) là phong cách thiết kế API phổ biến trên nền HTTP. App có thể gọi API để lấy danh sách thiết bị, trạng thái hiện tại, lịch sử event hoặc gửi yêu cầu điều khiển. REST dễ triển khai và dễ kiểm thử, phù hợp với các thao tác quản lý thông thường. Hạn chế của REST là mô hình request/response không tối ưu cho cập nhật realtime liên tục nếu dùng một mình; với yêu cầu realtime cao hơn, hệ thống có thể cần thêm WebSocket hoặc cơ chế push.

Database lưu trữ các dữ liệu bền vững của hệ thống. Các nhóm dữ liệu thường gặp gồm device registry, device state, event history, command lifecycle và automation rule. Device registry mô tả thiết bị trong hệ thống. Device state biểu diễn trạng thái hiện tại hoặc lịch sử trạng thái. Event history hỗ trợ truy vết sự kiện. Command lifecycle cho biết lệnh đã được tạo, gửi, xác nhận hay thất bại. Automation rule mô tả điều kiện và hành động tự động.

Trong đồ án, FastAPI được sử dụng cho REST API, SQLAlchemy hỗ trợ Object-Relational Mapping (ORM), PostgreSQL dùng cho dữ liệu production và Paho MQTT giúp Cloud kết nối với broker. Ở Chương 2, nhóm thực hiện chỉ trình bày vai trò của các thành phần này ở mức công nghệ nền; danh sách endpoint, schema chi tiết và xử lý nghiệp vụ được đặt ở các chương sau.

Bảng 2.4: Vai trò của các thành phần Cloud trong hệ thống

Thành phần	Vai trò lý thuyết
REST API	Cung cấp giao diện HTTP để App lấy dữ liệu và gửi yêu cầu điều khiển.
MQTT Client	Subscribe trạng thái từ Gateway và publish command xuống Gateway.
Database	Lưu thiết bị, trạng thái, sự kiện, command và automation rule.
ORM	Ảnh xạ giữa object trong code backend và bảng dữ liệu trong database.

2.11 Ứng dụng di động và quản lý trạng thái

Mobile App là lớp tương tác trực tiếp với người dùng cuối. Trong hệ thống Smart Building, người dùng cần xem trạng thái thiết bị, gửi lệnh điều khiển và theo dõi các event hoặc automation rule. App vì vậy chuyển thao tác của người dùng thành request gửi lên Cloud Backend, đồng thời trình bày lại dữ liệu mà Backend cung cấp.

Flutter là framework UI cross-platform, cho phép xây dựng giao diện trên nhiều nền tảng từ cùng một codebase [9]. Trong đồ án, App gọi REST API qua HTTP. Cách tổ chức này giữ App ở lớp người dùng và Cloud, không để App giao tiếp trực tiếp với Zigbee hoặc Gateway. Nhờ đó, App tập trung vào giao diện và trải nghiệm sử dụng, còn Gateway và Cloud xử lý các phần truyền thông thiết bị.

State management là cách App quản lý và cập nhật trạng thái giao diện. Khi danh sách thiết bị, trạng thái đèn hoặc log automation thay đổi, giao diện cần được cập nhật nhất quán. Provider là một cơ chế state management phổ biến trong Flutter, giúp tách dữ liệu/trạng thái khỏi widget hiển thị. Với đồ án này, state management giúp App phản ánh đúng trạng thái thiết bị sau khi người dùng gửi lệnh hoặc khi Gateway báo trạng thái mới.

Ưu điểm của việc để App làm việc qua REST API là kiến trúc rõ ràng và dễ kiểm thử. Hạn chế là cập nhật realtime có thể chưa mượt nếu chỉ dựa vào request/response. Vì vậy, trong phạm vi Chương 2, App được trình bày như lớp monitoring/control cho người dùng; chi tiết màn hình và luồng thao tác được trình bày ở chương thiết kế hoặc triển khai.

2.12 Bảo mật trong Zigbee và IoT

Bảo mật cần được xem xét ở nhiều lớp, từ thiết bị, mạng Zigbee, Gateway, MQTT Broker, Cloud Backend, cơ sở dữ liệu đến ứng dụng. Với Smart Building, sự cố bảo mật có thể dẫn đến mất dữ liệu hoặc làm sai lệch hành vi điều khiển thiết bị vật lý. Vì vậy, thiết kế phải kiểm soát cả thiết bị tham gia mạng, kênh truyền và quyền của người dùng.

Ở lớp Zigbee, các khái niệm cần chú ý gồm network key, application security và Trust Center. Network key giúp bảo vệ truyền thông trong cùng một mạng Zigbee. Trust Center liên quan đến việc quản lý thiết bị tham gia mạng và phân phối khóa trong một số mô hình bảo mật. Khi thiết bị join mạng, hệ thống cần kiểm soát thiết bị nào được phép tham gia và quá trình trao đổi khóa được thực hiện như thế nào.

Ở lớp MQTT và Cloud, broker cần xác thực client, giới hạn quyền trên topic và bảo vệ kênh truyền bằng Transport Layer Security (TLS). Cloud Backend cần xác thực người dùng, kiểm tra vai trò, giới hạn phạm vi nhà/thiết bị, kiểm tra dữ liệu đầu vào và bảo vệ cơ sở dữ liệu. Ứng dụng cũng cần lưu token trong vùng lưu trữ bảo mật thay vì tệp cấu hình thông thường.

Tăng cường bảo mật làm quá trình triển khai, vận hành và gỡ lỗi phức tạp hơn. Sự đánh đổi này cần được kiểm soát bằng cấu hình môi trường, test quyền truy cập và log có định danh rõ ràng. Báo cáo chỉ khẳng định các cơ chế đã có source và test tương ứng.

2.13 Công nghệ sử dụng trong đồ án

Các công nghệ trong đồ án được tổ chức theo kiến trúc nhiều lớp. Thiết bị Zigbee nằm ở tầng nhúng và mạng cục bộ. Gateway chịu trách nhiệm chuyển đổi giữa Zigbee và MQTT. MQTT Broker làm trung gian messaging. Cloud Backend quản lý dữ liệu và API. Mobile App là giao diện cho người dùng cuối. Cách chia lớp này giúp hệ thống dễ giải thích, dễ kiểm thử theo từng phần và thuận lợi hơn khi mở rộng.

Bảng 2.5: Tóm tắt công nghệ nền được sử dụng trong đồ án

Lớp	Công nghệ	Vai trò trong hệ thống
Device	EFR32, EmberZNet/Gecko SDK, Zigbee stack	Chạy firmware cho light, switch, occupancy sensor hoặc NCP.
Gateway	Z3GatewayHost, C, libmosquitto	Kết nối Zigbee local network với MQTT/Cloud.
Broker	Eclipse Mosquitto	Trung gian publish/subscribe giữa Cloud và Gateway.
Backend	FastAPI, SQLAlchemy, Paho MQTT	Cung cấp REST API, xử lý MQTT và quản lý dữ liệu/lệnh.
Database	PostgreSQL, SQLite trong một số ngữ cảnh test	Lưu thiết bị, trạng thái, sự kiện, command và automation.
App	Flutter, Dart, Provider, HTTP	Giao diện monitoring/control cho người dùng cuối.

Tóm lại, Chương 2 cung cấp phần cơ sở để người đọc hiểu các lựa chọn kỹ thuật chính của đồ án: Zigbee cho mạng thiết bị, Z3Gateway C cho lớp kết nối với coordinator radio, MQTT cho trao đổi trạng thái/lệnh, REST API và database cho lớp Cloud, Flutter cho giao diện người dùng. Các khái niệm này là tiền đề cho phần phân tích yêu cầu, thiết kế kiến trúc, triển khai và đánh giá ở các chương tiếp theo.

CHƯƠNG 3

PHÂN TÍCH YÊU CẦU VÀ THIẾT KẾ HỆ THỐNG

Chương này chuyển các cơ sở lý thuyết ở Chương 2 thành thiết kế cụ thể cho hệ thống của đồ án. Nội dung được trình bày theo luồng dữ liệu từ thiết bị Zigbee, qua Gateway và Message Queuing Telemetry Transport (MQTT), tới Cloud Backend, cơ sở dữ liệu và Mobile App. Cách sắp xếp này giúp người đọc xác định rõ dữ liệu bắt đầu ở đâu, được xử lý bởi thành phần nào và kết thúc tại đâu.

3.1 Phạm vi và giả định thiết kế

Hệ thống được thiết kế cho mô hình thử nghiệm trong một phòng hoặc phòng thí nghiệm. Phiên bản hiện tại quản lý một Gateway và ba loại thiết bị chính gồm đèn, công tắc và cảm biến chuyển động. Trong mô hình dữ liệu, ba loại thiết bị này lần lượt có giá trị light, switch và motion. Giá trị occupancy biểu diễn trạng thái phát hiện hiện diện của cảm biến chuyển động, không phải một loại thiết bị độc lập.

Thiết bị Zigbee không giao tiếp trực tiếp với Internet. Gateway đặt tại mạng cục bộ duy trì kết nối với bộ xử lý mạng EFR32, chuyển dữ liệu Zigbee thành bản tin MQTT và thực hiện chiều ngược lại đối với lệnh điều khiển. Cloud Backend cung cấp giao diện cho Mobile App, lưu dữ liệu bền vững và kiểm soát quyền truy cập. Mobile App là lớp tương tác với người dùng, không kết nối trực tiếp tới thiết bị hoặc Gateway.

3.2 Phân tích yêu cầu hệ thống

3.2.1 Tác nhân sử dụng hệ thống

Hệ thống có ba nhóm tác nhân. Người quản trị quản lý người dùng, thiết bị và quá trình thêm thiết bị. Người dùng điều khiển có thể xem và điều khiển thiết bị trong nhà được gán. Người xem chỉ được quan sát trạng thái và lịch sử sự kiện. Ngoài tác nhân con người, Gateway và các thiết bị Zigbee là tác nhân kỹ thuật vì chúng chủ động gửi trạng thái, sự kiện và phản hồi lệnh.

3.2.2 Yêu cầu chức năng

Bảng 3.1 tóm tắt các yêu cầu chức năng theo luồng vận hành của hệ thống.

Bảng 3.1: Các yêu cầu chức năng chính

Mã	Yêu cầu	Mô tả
FR-01	Quản lý mạng Zigbee	Gateway cho phép mở hoặc đóng mạng, nhận thiết bị tham gia và lưu thông tin nhận dạng thiết bị.
FR-02	Giám sát thiết bị	Hệ thống hiển thị danh sách thiết bị, trạng thái gần nhất và tình trạng trực tuyến.
FR-03	Điều khiển đèn	Người dùng có quyền gửi lệnh bật hoặc tắt đèn và theo dõi vòng đời của lệnh.
FR-04	Ghi nhận sự kiện	Công tắc, cảm biến chuyển động và Gateway có thể phát sinh sự kiện để Cloud lưu và hiển thị.
FR-05	Tự động hóa	Người dùng có quyền tạo và bật/tắt luật tự động hóa dựa trên sự kiện của thiết bị.
FR-06	Xác thực và phân quyền	Hệ thống xác thực người dùng, duy trì phiên đăng nhập và giới hạn thao tác theo vai trò và phạm vi nhà.
FR-07	Thêm thiết bị	Người dùng có quyền tạo phiên thêm thiết bị và theo dõi trạng thái của phiên.

Các yêu cầu trên tạo thành một vòng kín: thiết bị tham gia mạng, gửi dữ liệu, nhận lệnh và phản hồi; Cloud lưu lại diễn biến; Mobile App cung cấp điểm thao tác cho người dùng.

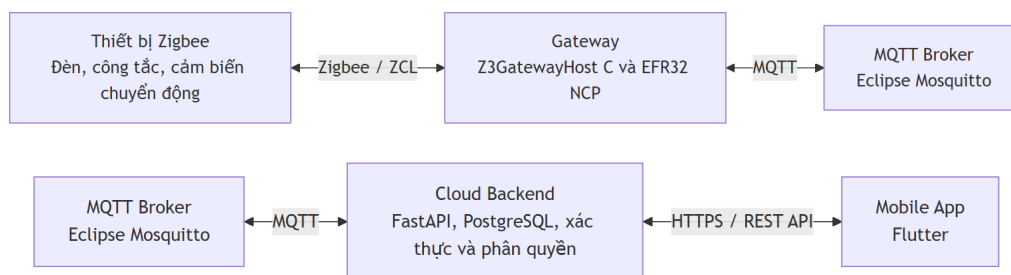
3.2.3 Yêu cầu phi chức năng

Hệ thống cần tách biệt rõ trách nhiệm giữa các lớp để lỗi ở một lớp không làm thay đổi hợp đồng dữ liệu của lớp khác. Bản tin phải có định danh thiết bị và thời điểm phát sinh. Lệnh điều khiển cần có trạng thái accepted, executed, failed hoặc timeout để người dùng không nhầm việc Cloud nhận yêu cầu với việc thiết bị đã thực thi.

Về bảo mật, Cloud phải từ chối yêu cầu không có phiên hợp lệ và kiểm tra phạm vi truy cập trước khi trả dữ liệu hoặc gửi lệnh. Về khả năng bảo trì, các module Gateway, router Cloud và repository của Mobile App cần có ranh giới rõ. Về kiểm thử, phần mềm Cloud và Mobile App phải có thể chạy test độc lập với phần cứng; các test xuyên suốt cần dùng thiết bị thật và lưu evidence.

3.3 Kiến trúc tổng thể hệ thống

Hình 3.1 trình bày các lớp chính và hai chiều dữ liệu của hệ thống.



Hình 3.1: Kiến trúc tổng thể và đường truyền dữ liệu của hệ thống

Theo chiều đi lên, thiết bị gửi trạng thái hoặc sự kiện qua mạng Zigbee. Gateway chuẩn hóa dữ liệu thành bản tin MQTT, Cloud kiểm tra và lưu vào cơ sở dữ liệu, sau đó Mobile App

đọc dữ liệu qua Application Programming Interface (API). Theo chiều đi xuống, Mobile App gửi yêu cầu tới Cloud; Cloud tạo bản ghi lệnh, phát bản tin MQTT; Gateway chuyển lệnh thành thao tác Zigbee và gửi phản hồi về Cloud.

3.4 Thiết kế thiết bị và trạng thái

3.4.1 Mô hình thiết bị

Bảng 3.2 mô tả hợp đồng tối thiểu của từng loại thiết bị. Mô hình này giúp Cloud và Mobile App xử lý thiết bị theo loại mà không cần hiểu chi tiết firmware.

Bảng 3.2: Mô hình các loại thiết bị phiên bản hiện tại

Loại	Vai trò Zigbee	Trạng thái chính	Hành vi chính
light	Router, On/Off Server	power, level, reachable	Nhận lệnh bật/tắt và báo cáo trạng thái.
switch	Router, On/Off Client	reachable, tùy chọn battery	Phát sinh sự kiện khi người dùng nhấn nút.
motion	End Device, Occupancy Sensing Server	occupancy, reachable, tùy chọn battery	Báo sự kiện khi trạng thái hiện diện thay đổi.

Định danh EUI-64 được dùng để liên hệ thiết bị vật lý với bản ghi logic trong hệ thống. Địa chỉ mạng ngắn chỉ có ý nghĩa trong phiên mạng hiện tại, vì vậy không được dùng làm định danh bền vững.

3.4.2 Trạng thái, sự kiện và lệnh

Trạng thái mô tả giá trị hiện tại của thiết bị, ví dụ đèn đang bật. Sự kiện mô tả một việc vừa xảy ra, ví dụ công tắc được nhấn hoặc cảm biến thay đổi giá trị hiện diện. Lệnh mô tả yêu cầu thay đổi trạng thái, ví dụ bật đèn. Việc tách ba khái niệm này giúp hệ thống tránh dùng một bản ghi cho nhiều mục đích.

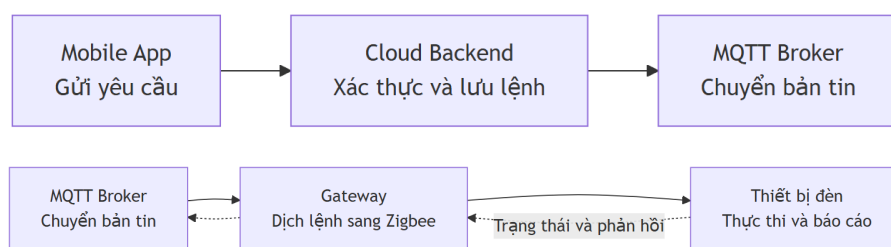
3.5 Luồng dữ liệu hệ thống

3.5.1 Luồng đi lên: thiết bị báo cáo lên Cloud

Thiết bị gửi báo cáo thuộc tính hoặc sự kiện tới Coordinator. Gateway nhận callback từ Zigbee stack, xác định loại thiết bị, thêm định danh và đóng gói dữ liệu theo hợp đồng MQTT. Cloud nhận bản tin, cập nhật thời điểm nhìn thấy thiết bị, lưu trạng thái hoặc sự kiện và cung cấp dữ liệu cho Mobile App.

3.5.2 Luồng đi xuống: người dùng gửi lệnh điều khiển

Hình 3.2 mô tả luồng điều khiển đèn ở mức thiết kế.



Hình 3.2: Luồng thiết kế cho lệnh điều khiển đèn

Cloud trả trạng thái `accepted` khi đã nhận và phát lệnh, không đồng nghĩa thiết bị đã thực thi. Mobile App tiếp tục truy vấn trạng thái lệnh. Khi Gateway gửi phản hồi, Cloud cập nhật lệnh thành `executed` hoặc `failed`; worker thời gian chờ chuyển lệnh quá hạn thành `timeout`.

3.5.3 Luồng tự động hóa

Một luật tự động hóa gồm nguồn kích hoạt, điều kiện tùy chọn và hành động. Ví dụ, sự kiện `occupancy_changed` của cảm biến chuyển động có thể kích hoạt hành động bật đèn. Cloud quản lý cấu hình luật; Gateway có thể thực hiện luật cục bộ để giảm phụ thuộc vào đường truyền Internet. Sự đánh đổi là luật cần được đồng bộ và có phiên bản để Cloud và Gateway không dùng hai cấu hình khác nhau.

3.6 Thiết kế hợp đồng MQTT

MQTT topic sử dụng tiền tố `sb/v1/{tenant}/{site}/{gateway}`. Ba thành phần này tạo không gian tên cho đơn vị sử dụng, địa điểm và Gateway. Payload sử dụng envelope `sb.v1` để các bên kiểm tra phiên bản hợp đồng.

Bảng 3.3 trình bày các nhóm topic chính.

Bảng 3.3: Các nhóm topic MQTT chính

Mẫu topic	Chiều dữ liệu	Mục đích
<code>devices/{type}/{id}/reported</code>	Gateway lên Cloud	Báo trạng thái hiện tại.
<code>devices/{type}/{id}/event</code>	Gateway lên Cloud	Báo sự kiện của thiết bị.
<code>devices/{type}/{id}/registry</code>	Gateway lên Cloud	Báo định danh và phân loại thiết bị.
<code>commands/{id}/request</code>	Cloud xuống Gateway	Yêu cầu thực thi lệnh.
<code>commands/{id}/reply</code>	Gateway lên Cloud	Báo kết quả hoặc nguyên nhân lỗi.
<code>gateway/online, gateway/health</code>	Gateway lên Cloud	Báo kết nối và tình trạng Gateway.

Việc dùng topic theo thiết bị giúp Cloud đăng ký nhận theo mẫu. Tuy nhiên, quyền truy cập của MQTT Broker phải giới hạn đúng tiền tố `tenant`, `site` và `gateway` để một Gateway không đọc hoặc ghi dữ liệu của Gateway khác.

3.7 Thiết kế Gateway biên

Gateway được thiết kế là ứng dụng Host viết bằng C, sử dụng `libmosquitto` để làm việc với MQTT và giao tiếp với EFR32 NCP qua EmberZNet Serial Protocol (EZSP) trên Asynchronous Serial Host (ASH). Thiết kế này không dùng một tiến trình Python làm cầu nối trung gian.

Callback MQTT chỉ kiểm tra sơ bộ và đưa lệnh vào hàng đợi. Vòng lặp chính lấy lệnh, phân tích envelope, xác định đích, gọi module điều khiển phù hợp và phát phản hồi. Ở chiều ngược lại, callback của Zigbee stack chuyển báo cáo thuộc tính và sự kiện cho module MQTT. Cách dùng hàng đợi tránh thực hiện thao tác Zigbee dài bên trong luồng callback mạng.

Gateway còn duy trì registry thiết bị để ánh xạ EUI-64, địa chỉ mạng và loại thiết bị. Khi thiết bị mới tham gia, Gateway thực hiện discovery, phát bản tin registry và tiếp tục nhận báo cáo trạng thái. Nếu phân loại thất bại hoặc thiết bị ngoại tuyến, Gateway cần trả lý do rõ thay vì tự gán sai loại.

3.8 Thiết kế Cloud Backend, cơ sở dữ liệu và phân quyền

Cloud Backend là điểm trung tâm giữa Mobile App và Gateway. Thành phần này cung cấp API, duy trì kết nối MQTT, lưu dữ liệu, quản lý phiên đăng nhập và kiểm tra quyền. Trong đồ án, App, cơ sở dữ liệu và phân quyền không được xem là các hướng nghiên cứu độc lập; chúng hỗ trợ chứng minh vòng đời vận hành xuyên suốt của hệ thống Zigbee.

3.8.1 REST API

Representational State Transfer (REST) là cách tổ chức API theo tài nguyên và thao tác Hypertext Transfer Protocol (HTTP). Các router được chia theo nhóm thiết bị, lệnh, sự kiện, Gateway, tự động hóa, phiên thêm thiết bị và xác thực. Create, Read, Update, Delete (CRUD) là bốn thao tác tạo, đọc, cập nhật và xóa; nhóm API tự động hóa hỗ trợ các thao tác này trong phạm vi quyền của người dùng.

Bảng 3.4: Các nhóm REST API chính

Nhóm	Endpoint tiêu biểu	Mục đích
Xác thực	/auth/login, /auth/refresh, /auth/me	Đăng nhập, làm mới phiên và lấy người dùng hiện tại.
Thiết bị	GET /api/devices/, GET /api/devices/{id}/state	Đọc thiết bị và trạng thái.
Lệnh	POST /api/devices/{id}/command, GET /api/commands/{id}	Tạo lệnh và theo dõi kết quả.
Sự kiện	GET /api/events/	Đọc lịch sử sự kiện trong phạm vi được phép.
Tự động hóa	/api/automations, /api/automation-events	Quản lý luật và lịch sử thực thi.
Thêm thiết bị	/api/provisioning/sessions	Tạo và theo dõi phiên thêm thiết bị.

3.8.2 Cơ sở dữ liệu

Cơ sở dữ liệu quan hệ lưu trạng thái bền vững của hệ thống. Báo cáo nhóm các bảng theo trách nhiệm thay vì liệt kê toàn bộ cột.

Bảng 3.5: Các nhóm bảng dữ liệu chính

Nhóm bảng	Dữ liệu lưu trữ
users, auth_refresh_tokens	Người dùng, vai trò, mật khẩu đã băm, phiên làm mới và trạng thái hoạt động.
homes, rooms, devices	Cấu trúc nhà, phòng, thiết bị, loại thiết bị và thông tin trực tuyến.
device_states, events	Trạng thái được báo cáo và lịch sử sự kiện.
commands	Lệnh, đích, thời gian chờ, trạng thái và nguyên nhân lỗi.
automations, automation_events	Cấu hình luật và lịch sử đồng bộ hoặc thực thi.
factory_devices, provisioning_sessions	Thiết bị xuất xưởng và vòng đời phiên thêm thiết bị.

3.8.3 Xác thực và phân quyền theo vai trò

Xác thực trả lời câu hỏi người gửi yêu cầu là ai. Phân quyền trả lời câu hỏi người đó được phép làm gì. Sau khi đăng nhập, Cloud cấp access token dùng cho các yêu cầu API ngắn hạn và refresh token dùng để tạo phiên mới khi access token hết hạn. Mobile App lưu phiên bằng vùng lưu trữ bảo mật của hệ điều hành.

Role-Based Access Control (RBAC) là cơ chế gán quyền theo vai trò. Hệ thống hiện có ba vai trò admin, parent và viewer. Bảng 3.6 trình bày quyền chính.

Bảng 3.6: Ma trận quyền theo vai trò

Chức năng	Admin	Parent	Viewer
Xem thiết bị và trạng thái	Có	Có, trong nhà được gán	Có, trong nhà được gán
Điều khiển thiết bị	Có	Có, trong nhà được gán	Không
Quản lý luật tự động hóa	Có	Có, trong nhà được gán	Không
Mở mạng hoặc thêm thiết bị	Có	Theo endpoint được cấp	Không
Quản lý người dùng và nhân xuất xưởng	Có	Không	Không

3.8.4 Kiểm soát phạm vi nhà và thiết bị

Kiểm tra vai trò là chưa đủ. Một người dùng parent của nhà A không được điều khiển thiết bị thuộc nhà B. Cloud lần theo quan hệ người dùng–nhà–phòng–thiết bị trước khi trả dữ liệu hoặc tạo lệnh. Với người dùng không phải quản trị viên, danh sách sự kiện và luật tự động hóa cũng được lọc theo các thiết bị nhìn thấy trong nhà được gán.

3.9 Thiết kế Mobile App

3.9.1 Vai trò của Mobile App

Mobile App chuyển thao tác của người dùng thành yêu cầu API và trình bày dữ liệu Cloud trả về. App không tự quyết định quyền; Cloud vẫn là nơi kiểm tra access token, vai trò và phạm vi. Cách này ngăn việc sửa giao diện phía người dùng để vượt qua kiểm soát quyền.

3.9.2 Các nhóm màn hình chính

Ứng dụng gồm màn hình đăng nhập, trang tổng quan, danh sách và chi tiết thiết bị, lịch sử sự kiện, tự động hóa, thêm thiết bị và cài đặt. Các màn hình sử dụng repository để tách giao diện khỏi chi tiết gọi HTTP. Provider quản lý trạng thái giao diện và thông báo cho widget khi dữ liệu thay đổi.

3.9.3 Luồng gọi API từ Mobile App

Khi khởi động, App đọc phiên từ vùng lưu trữ bảo mật. Nếu access token hết hạn nhưng refresh token còn hiệu lực, App gọi API làm mới phiên. Sau khi đăng nhập, App lấy danh sách thiết bị và trạng thái, hiển thị cho người dùng, rồi gửi lệnh khi người dùng có quyền thao tác. Đối với lệnh điều khiển, App truy vấn trạng thái lệnh cho tới khi nhận trạng thái kết thúc hoặc hết thời gian chờ.

3.10 Các quyết định thiết kế và sự đánh đổi

Bảng 3.7 tổng hợp các quyết định ảnh hưởng trực tiếp tới khả năng vận hành và bảo trì.

Bảng 3.7: Các quyết định thiết kế và sự đánh đổi

Quyết định	Lợi ích	Sự đánh đổi
Dùng Gateway C kết nối MQTT trực tiếp	Giảm một tiến trình trung gian và bám sát Zigbee host stack.	Cần quản lý bộ nhớ, hàng đợi và lỗi đồng thời cẩn thận.
Dùng MQTT giữa Cloud và Gateway	Hỗ trợ hai chiều, tách bên gửi và bên nhận.	Cần quản lý topic, quyền truy cập, bản tin giữ lại và trạng thái kết nối.
Lưu lệnh trước khi phát MQTT	Theo dõi được vòng đời và lỗi thời gian chờ.	Tăng số thao tác cơ sở dữ liệu và cần đồng bộ phản hồi.
Đặt kiểm tra quyền tại Cloud	Không phụ thuộc vào việc giao diện có ẩn nút hay không.	Mỗi endpoint phải kiểm tra cả vai trò và phạm vi.
Mobile App gọi REST API và truy vấn trạng thái lệnh	Dễ triển khai và kiểm thử.	Trạng thái không cập nhật tức thời như cơ chế đẩy dữ liệu.

Các quyết định trên ưu tiên tính rõ ràng và khả năng kiểm chứng trong phạm vi đồ án. Chúng không nhằm giải quyết toàn bộ bài toán triển khai nhiều tòa nhà, nhưng giữ được ranh giới đủ rõ để mở rộng trong các phiên bản sau.

CHƯƠNG 4

TRIỂN KHAI HỆ THỐNG

Chương này trình bày cách các khối thiết kế ở Chương 3 được hiện thực trong mã nguồn. Mỗi phần trả lời năm câu hỏi: thành phần nhận dữ liệu gì, xử lý ở đâu, tạo ra dữ liệu gì, liên hệ với thành phần kế tiếp như thế nào và có thể kiểm tra bằng evidence nào. Cách trình bày này giúp người đọc không cần biết sâu về Zigbee hoặc lập trình nhúng vẫn theo dõi được luồng vận hành của hệ thống.

4.1 Cách đọc phần triển khai

Hệ thống được triển khai theo bốn lớp. Lớp thiết bị biến tác động vật lý thành trạng thái hoặc sự kiện Zigbee. Lớp Gateway chuyển đổi giữa Zigbee và Message Queuing Telemetry Transport (MQTT). Lớp Cloud lưu dữ liệu, kiểm soát quyền và cung cấp Application Programming Interface (API). Lớp Mobile App cung cấp giao diện cho người dùng.

Trong mỗi lớp, đầu vào và đầu ra phải giữ đúng hợp đồng đã thiết kế. Ví dụ, cảm biến chuyển động có thể phát trạng thái occupancy, nhưng loại thiết bị vẫn là motion. Tương tự, Cloud trả trạng thái lệnh accepted khi đã tiếp nhận yêu cầu; chỉ phản hồi executed từ Gateway mới chứng minh thiết bị đã thực thi.

4.2 Môi trường triển khai và quản lý mã nguồn

Bảng 4.1 trình bày các vùng mã nguồn chính. Các đường dẫn được lấy từ repository hiện tại của hệ thống.

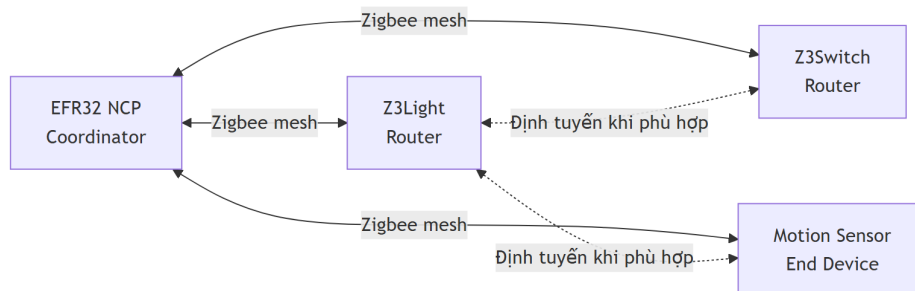
Bảng 4.1: Bản đồ thành phần và mã nguồn triển khai

Thành phần	Công nghệ chính	Vị trí mã nguồn
Thiết bị Zigbee	Gecko Software Development Kit (GSDK), EmberZNet, ngôn ngữ C	end_devices/ncp-uart-hw-fresh/, end_devices/Z3Light/, end_devices/Z3Switch/, end_devices/Z3_Occupancy_Sensor/
Gateway	Z3GatewayHost, C, libmosquitto, EmberZNet Serial Protocol (EZSP)	gateway/Z3GatewayHost/app/
Cloud Backend	Python, FastAPI, SQLAlchemy, Paho MQTT	cloud/app/, cloud/app/routers/
Cơ sở dữ liệu	PostgreSQL trong môi trường demo, SQLite cho một số test	cloud/app/models.py, cloud/app/database.py
Mobile App	Flutter, Dart, Provider, vùng lưu trữ token bảo mật	mobile_app/lib/
Triển khai	Docker Compose và cấu hình môi trường	deploy/

Nhóm sử dụng Jira để chia và theo dõi công việc. Git là hệ thống quản lý phiên bản phân tán dùng để ghi lại lịch sử thay đổi mã nguồn. GitHub là dịch vụ lưu repository từ xa, hỗ trợ chia sẻ branch, review và hợp nhất thay đổi. Ba công cụ này không tham gia luồng chạy của hệ thống, nhưng giúp hai thành viên kiểm soát phạm vi công việc và truy vết thay đổi.

4.3 Triển khai mạng Zigbee và thiết bị

Hình 4.1 tập trung vào các node trong mạng cục bộ. Sơ đồ được tách khỏi kiến trúc Cloud để người đọc quan sát rõ vai trò của từng thiết bị.



Hình 4.1: Các node Zigbee được triển khai trong mô hình thử nghiệm

EFR32 Network Co-Processor (NCP) giữ vai trò Coordinator và xử lý phần radio. Ứng dụng Host trên Gateway gửi yêu cầu tới NCP qua giao tiếp nối tiếp. Đèn và công tắc hoạt động như Router; cảm biến chuyển động hoạt động như End Device để phù hợp với bài toán thiết bị cảm biến công suất thấp.

4.3.1 Coordinator và NCP

Firmware NCP khởi tạo Zigbee stack, cung cấp giao diện EZSP và cho phép Host mở hoặc đóng mạng để thiết bị tham gia. Đầu vào của NCP là yêu cầu từ Host và khung Zigbee

nhận qua radio. Đầu ra là callback hoặc phản hồi EZSP cho Host. Evidence phù hợp gồm log tạo mạng, địa chỉ mạng của Coordinator và log permit join.

4.3.2 Thiết bị đèn

Thiết bị đèn triển khai On/Off Server của Zigbee Cluster Library (ZCL), tức thư viện chuẩn hóa cluster, thuộc tính và lệnh cho thiết bị Zigbee. Khi nhận lệnh bật hoặc tắt, firmware cập nhật đầu ra điều khiển đèn và báo cáo lại thuộc tính. Đầu vào là lệnh ZCL; đầu ra là trạng thái power, mức sáng nếu có và trạng thái có thể kết nối.

4.3.3 Thiết bị công tắc

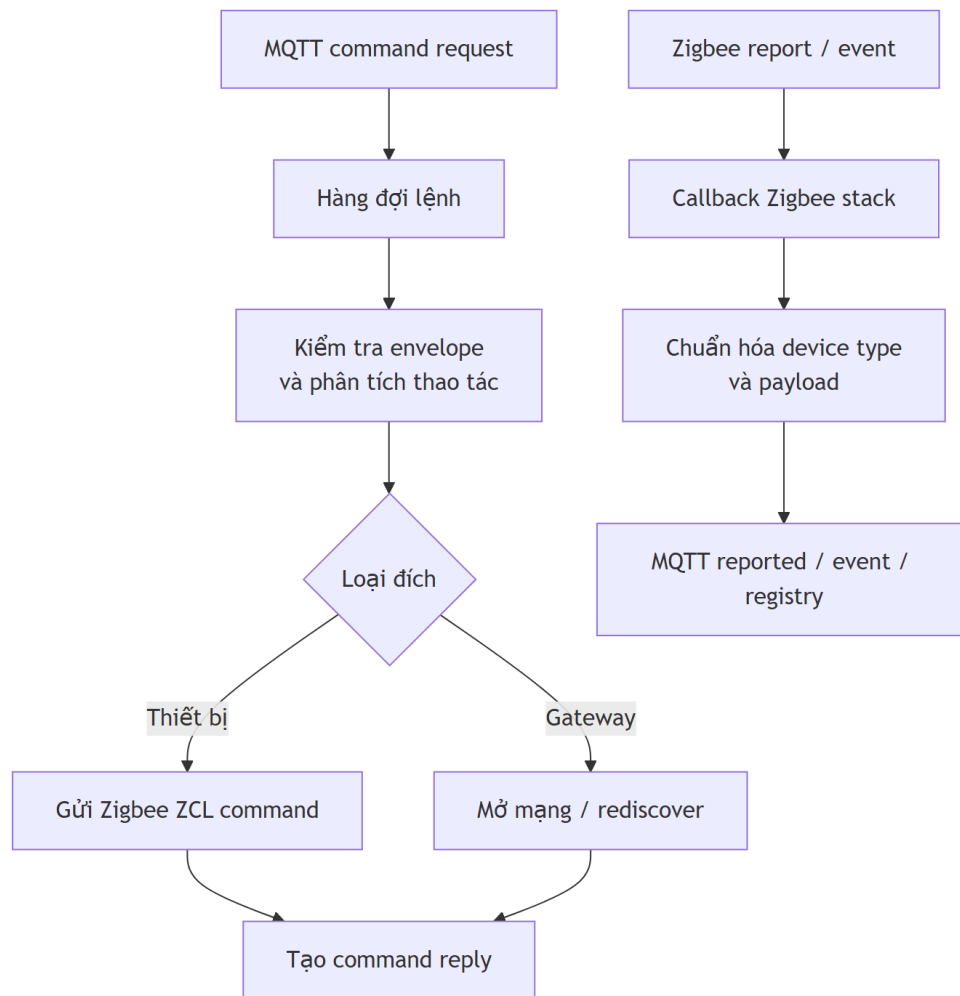
Công tắc triển khai vai trò On/Off Client. Thao tác nút nhấn được chuyển thành sự kiện để Gateway và luật tự động hóa xử lý. Công tắc không được coi là đèn và không nhận lệnh điều khiển công suất từ Cloud. Evidence phù hợp gồm log nút nhấn, bản tin sự kiện MQTT và hành vi của thiết bị đích khi có luật.

4.3.4 Cảm biến chuyển động

Cảm biến triển khai Occupancy Sensing Server. Firmware đọc trạng thái cảm biến, cập nhật thuộc tính hiện diện và gửi báo cáo khi giá trị thay đổi. Đầu ra của lớp thiết bị là occupancy; Gateway đóng gói thành sự kiện occupancy_changed và trạng thái của thiết bị loại motion.

4.4 Triển khai Gateway

Gateway là điểm nối giữa hai môi trường có mô hình xử lý khác nhau. Zigbee hoạt động theo callback của stack và địa chỉ thiết bị trong mạng cục bộ. MQTT hoạt động theo topic và payload JavaScript Object Notation (JSON), tức định dạng dữ liệu dạng cặp khóa–giá trị. Hình 4.2 mô tả các bước xử lý chính trong Gateway.



Hình 4.2: Luồng xử lý dữ liệu trong Gateway

4.4.1 Kết nối MQTT và hàng đợi lệnh

Module `app_mqtt.c` đọc cấu hình broker, thông tin tenant, site và Gateway từ biến môi trường. Trong chế độ production, module yêu cầu thông tin xác thực và đường dẫn chứng thư nếu bật Transport Layer Security (TLS), tức cơ chế bảo vệ kênh truyền. Sau khi kết nối, Gateway đăng ký topic lệnh và phát định kỳ trạng thái hoạt động.

Callback nhận bản tin không điều khiển Zigbee trực tiếp. Bản tin được đưa vào ring buffer có khóa đồng bộ; vòng lặp chính lấy từng phần tử để xử lý. Giới hạn của cách làm này là hàng đợi có kích thước hữu hạn, nhưng đổi lại callback MQTT kết thúc nhanh và không chặn luồng mạng.

4.4.2 Phân tích và thực thi lệnh

Gateway kiểm tra phiên bản envelope, định danh lệnh, thao tác và đích. Lệnh điều khiển đèn được chuyển thành ZCL On/Off command. Lệnh dành cho Gateway, như mở mạng hoặc rediscover thiết bị, được chuyển tới module quản lý mạng. Sau khi xử lý, Gateway phát command reply với trạng thái và lý do lỗi nếu có.

4.4.3 Discovery và registry thiết bị

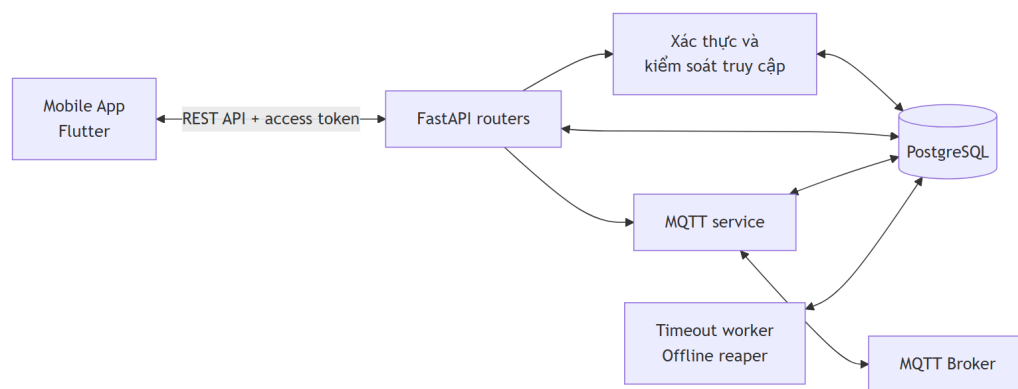
Khi một node tham gia mạng, Gateway lấy EUI-64, địa chỉ mạng và Simple Descriptor để phân loại thiết bị. Kết quả được lưu trong registry cục bộ và phát lên topic registry. Cloud sử dụng dữ liệu này để tạo hoặc cập nhật bản ghi thiết bị. Nếu một thiết bị chưa thể phân loại, hệ thống cần giữ trạng thái chưa xác định và thử lại, không tự gán thành một loại đã biết.

4.4.4 Trạng thái, sự kiện và luật cục bộ

Callback báo cáo thuộc tính tạo bản tin reported state. Callback nút nhấn hoặc cảm biến tạo bản tin event. Rule engine có thể dùng sự kiện công tắc hoặc chuyển động để gửi lệnh tới đèn ngay tại Gateway. Xử lý cục bộ giảm độ trễ và vẫn hoạt động khi mất kết nối Cloud, nhưng cấu hình luật phải được đồng bộ để tránh khác biệt giữa Cloud và Gateway.

4.5 Triển khai Cloud Backend, cơ sở dữ liệu và phân quyền

Hình 4.3 trình bày các module Cloud và điểm nối với Mobile App. Sơ đồ chỉ tập trung vào xử lý phần mềm phía trên Gateway.



Hình 4.3: Các module Cloud Backend và luồng trao đổi với Mobile App

4.5.1 Vòng đời ứng dụng và router

Tệp `cloud/app/main.py` tạo FastAPI application, khởi tạo cơ sở dữ liệu, kết nối MQTT và chạy hai background worker. Command timeout worker đánh dấu lệnh quá hạn. Device offline reaper đánh dấu thiết bị ngoại tuyến khi `last_seen_at` vượt ngưỡng cấu hình. Các router xác thực, thiết bị, lệnh, sự kiện, Gateway, tự động hóa và thêm thiết bị được đăng ký độc lập.

4.5.2 Mô hình dữ liệu và vòng đời lệnh

SQLAlchemy Object-Relational Mapping (ORM) là lớp ánh xạ object trong Python sang bảng quan hệ. Các model chính gồm người dùng, nhà, phòng, thiết bị, trạng thái, sự kiện, lệnh, luật tự động hóa và phiên thêm thiết bị. Khi Mobile App gửi lệnh, Cloud kiểm tra quyền, dịch thao tác sang hợp đồng Gateway, lưu bản ghi `accepted`, rồi mới phát MQTT. Phản hồi từ Gateway cập nhật cùng bản ghi thành trạng thái kết thúc.

4.5.3 Đăng nhập và quản lý phiên

Router xác thực kiểm tra tên đăng nhập, trạng thái người dùng và mật khẩu đã bấm. Khi đăng nhập thành công, Cloud cấp access token và refresh token. Refresh token được lưu dưới dạng giá trị bấm trong bảng riêng và có thể bị thu hồi khi đăng xuất. Thiết kế này tránh lưu token làm mới ở dạng rõ trong cơ sở dữ liệu.

4.5.4 Phân quyền và phạm vi truy cập

Dependency của FastAPI lấy người dùng hiện tại từ access token. Các endpoint thay đổi dữ liệu yêu cầu vai trò phù hợp. Sau kiểm tra vai trò, module `access_control.py` tiếp tục kiểm tra thiết bị hoặc luật tự động hóa có thuộc nhà của người dùng hay không. Vì vậy, việc có vai trò parent không tự động cho phép truy cập mọi thiết bị trong hệ thống.

4.5.5 Nhận dữ liệu MQTT

MQTT service đăng ký các topic reported, telemetry, event, registry, command reply, gateway health và automation. Mỗi loại bản tin được chuyển tới một handler riêng. Handler dùng phiên cơ sở dữ liệu bất đồng bộ để lưu dữ liệu, cập nhật `last_seen_at` và đánh dấu thiết bị trực tuyến. Payload rỗng trên retained topic được hiểu là yêu cầu xóa bản tin giữ lại và không được phân tích như JSON.

4.6 Triển khai Mobile App

Mobile App được tổ chức thành ba vùng: model miền nghiệp vụ, repository dữ liệu và giao diện. Repository từ xa gọi API thật; repository mô phỏng hỗ trợ kiểm thử giao diện. Provider phân phối view model cho các màn hình và cập nhật widget khi trạng thái thay đổi.

4.6.1 Đăng nhập và lưu phiên

Repository xác thực gọi các API đăng nhập, làm mới phiên, lấy thông tin người dùng và đăng xuất. Access token được gắn vào header của yêu cầu cần xác thực. Lớp lưu token bảo mật giữ phiên trong vùng lưu trữ của hệ điều hành. Khi App khởi động, auth gate thử khôi phục phiên trước khi quyết định hiển thị màn hình đăng nhập hay giao diện chính.

4.6.2 Giám sát và điều khiển thiết bị

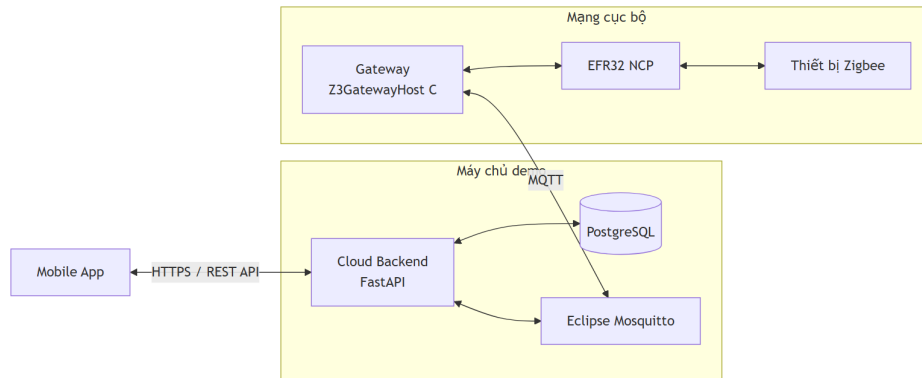
RemoteDeviceRepository lấy danh sách thiết bị, trạng thái mới nhất và sự kiện. Khi người dùng bật hoặc tắt đèn, repository gửi yêu cầu tạo lệnh. View model hiển thị trạng thái chờ, truy vấn command status và tải lại dữ liệu sau khi lệnh kết thúc. App không tự đổi trạng thái đèn thành công trước khi nhận dữ liệu từ Cloud.

4.6.3 Tự động hóa và thêm thiết bị

Repository tự động hóa cung cấp các thao tác tạo, sửa, bật, tắt và xóa luật. Repository thêm thiết bị tạo phiên provisioning, theo dõi trạng thái và gửi các yêu cầu liên quan tới Gateway thông qua Cloud. Quyền thực hiện vẫn do Cloud quyết định; việc hiển thị nút trong App chỉ giúp giao diện phù hợp với vai trò.

4.7 Triển khai môi trường demo

Hình 4.4 mô tả các container và thành phần chạy trong môi trường demo. Sơ đồ không biểu diễn chi tiết hạ tầng mạng bên ngoài để giữ kích thước vừa phải.

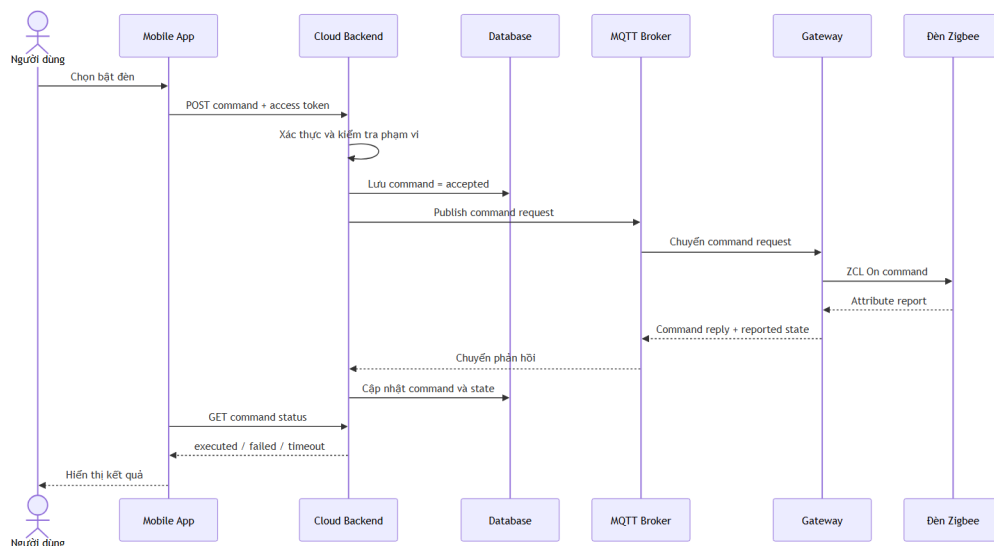


Hình 4.4: Các thành phần chính trong môi trường demo

Docker Compose khởi chạy Cloud Backend, PostgreSQL và MQTT Broker trên máy chủ. Gateway chạy tại mạng cục bộ nơi có EFR32 NCP và kết nối tới broker bằng thông tin cấu hình môi trường. Mobile App gọi địa chỉ API của Cloud. Khi triển khai qua mạng công cộng, cấu hình HTTPS, TLS, credential và Access Control List (ACL), tức danh sách quy định quyền trên MQTT topic, phải được bật và kiểm tra độc lập.

4.8 Luồng điều khiển đèn xuyên suốt

Hình 4.5 mô tả trình tự của một lệnh bật đèn. Sequence diagram được tách khỏi sơ đồ kiến trúc để chỉ thể hiện thứ tự thời gian.



Hình 4.5: Trình tự xử lý lệnh bật đèn từ Mobile App tới thiết bị

Mobile App gửi yêu cầu có access token. Cloud xác thực người dùng, kiểm tra quyền trên thiết bị và tạo bản ghi lệnh. MQTT Broker chuyển request tới Gateway. Gateway gửi ZCL On command tới đèn, sau đó phát reply và trạng thái được báo cáo. Cloud cập nhật lệnh và trạng

thái; Mobile App truy vấn lại để hiển thị kết quả. Nếu không có reply trước thời hạn, timeout worker chuyển lệnh sang timeout.

4.9 Giới hạn triển khai hiện tại

Phiên bản hiện tại tập trung vào một Gateway và số lượng thiết bị nhỏ trong môi trường thử nghiệm. Mobile App dùng cơ chế truy vấn lại dữ liệu nên độ cập nhật phụ thuộc chu kỳ gọi API. Registry và rule engine cần thêm thử nghiệm dài hạn khi số thiết bị và số luật tăng. Việc triển khai production còn cần giám sát tập trung, sao lưu cơ sở dữ liệu, xoay vòng credential và kiểm thử phục hồi sau lỗi.

Các giới hạn trên là ranh giới của phần triển khai hiện tại, không được xem là kết quả thất bại. Chương 5 sẽ chuyển từng yêu cầu thành test case và chỉ ghi nhận kết quả khi có evidence tương ứng.

4.10 Tổng kết chương

Chương 4 đã mô tả cách dữ liệu được tạo tại thiết bị Zigbee, chuyển qua Gateway, lưu tại Cloud và hiển thị trên Mobile App. Việc tách theo đầu vào, xử lý và đầu ra cho thấy ranh giới trách nhiệm của từng lớp. Các sơ đồ trong chương được tạo từ mã Mermaid lưu cùng repository để có thể chỉnh sửa và render lại khi kiến trúc thay đổi.

CHƯƠNG 5

KIỂM THỬ VÀ ĐÁNH GIÁ

Chương này kiểm tra hệ thống theo hai lớp evidence. Test phần mềm được chạy tự động trên source hiện tại và có số lượng pass/fail cụ thể. Test phần cứng và test xuyên suốt cần log Gateway, bản tin MQTT, phản hồi API, ảnh hoặc video của thiết bị thật. Những trường hợp chưa có evidence không được ghi là đạt.

5.1 Mục tiêu và nguyên tắc kiểm thử

Mục tiêu kiểm thử là xác nhận từng lớp hoạt động đúng hợp đồng và xác nhận dữ liệu có thể đi xuyên suốt từ thiết bị tới người dùng. Một test phần mềm đạt không thay thế cho test phần cứng. Ví dụ, test Cloud có thể chứng minh endpoint tạo đúng MQTT payload, nhưng không chứng minh đèn vật lý đã bật.

Mỗi test case gồm đầu vào, thao tác, kết quả mong đợi, kết quả thực tế, đánh giá và evidence. Kết quả chỉ được ghi *Đạt* khi evidence hiện có chứng minh đúng phạm vi của test. Nếu chưa chạy hoặc chưa lưu bằng chứng, ô kết quả được đánh dấu TODO.

5.2 Môi trường kiểm thử

Bảng 5.1 mô tả các môi trường cần dùng. Thông tin phần cứng cần được kiểm tra lại tại thời điểm chạy test thực nghiệm.

Bảng 5.1: Môi trường dùng cho các nhóm kiểm thử

Nhóm	Môi trường
Thiết bị Zigbee	EFR32 NCP, Z3Light, Z3Switch và cảm biến chuyển động; firmware được build từ các thư mục <code>end_devices/</code> .
Gateway	Z3GatewayHost C, kết nối serial tới NCP và kết nối MQTT tới broker.
Cloud	Python, FastAPI, cơ sở dữ liệu test và <code>pytest</code> .
Mobile App	Flutter test runner và widget test trong thư mục <code>mobile_app/test/</code> .
Xuyên suốt	Mobile App, Cloud Backend, MQTT Broker, Gateway và thiết bị thật cùng hoạt động.

Các test phần mềm trong chương được chạy ngày 12/06/2026 trên source tại `D:/CODE/zigbee-ble-orchestration-platform`. Kết quả phần cứng phải được bổ sung theo đúng cấu hình thiết bị được dùng trong buổi đo.

5.3 Danh sách test case

Bảng 5.2 tách test theo từng thiết bị và các lớp hỗ trợ. Bảng sử dụng nhiều cột để giữ cùng một cấu trúc đánh giá cho cả test tự động và test thực nghiệm.

Bảng 5.2: Test case cho thiết bị, Gateway, Cloud và Mobile App

Mã	Tên	Tóm tắt	Thao tác chi tiết	Đầu vào	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Evidence
HW-01	Tạo mạng và join	Kiểm tra Coordinator và thiết bị tham gia mạng.	Khởi động NCP/Gateway, mở mạng, bật từng thiết bị và ghi lại địa chỉ.	Thời gian permit join.	Thiết bị join, có EUI-64, địa chỉ mạng và registry đúng loại.	TODO: Chạy lại với thiết bị thật.	TODO: Chưa đánh giá.	TODO: Log join và ảnh thiết bị.
HW-02	Đèn nhận lệnh	Kiểm tra End Device đèn.	Gửi ZCL On, Off từ Gateway và quan sát trạng thái báo cáo.	Lệnh bật/tắt.	Đèn đổi trạng thái và phát reported state tương ứng.	TODO: Bổ sung kết quả đo.	TODO: Chưa đánh giá.	TODO: Log ZCL, MQTT và ảnh đèn.
HW-03	Công tắc phát sự kiện	Kiểm tra End Device công tắc.	Nhấn nút, theo dõi callback Zigbee và topic event.	Một lần nhấn nút.	Gateway nhận sự kiện đúng thiết bị, Cloud lưu event.	TODO: Bổ sung kết quả đo.	TODO: Chưa đánh giá.	TODO: Log nút nhấn và API event.
HW-04	Cảm biến chuyển động	Kiểm tra thiết bị loại motion.	Tạo và kết thúc chuyển động trước cảm biến, theo dõi thuộc tính.	Có/không có chuyển động.	occupancy thay đổi; loại thiết bị vẫn là motion.	TODO: Bổ sung kết quả đo.	TODO: Chưa đánh giá.	TODO: Log attribute report và API state.
HW-05	App điều khiển đèn	Kiểm tra luồng xuyên suốt.	Đăng nhập, chọn đèn, gửi lệnh bật và chờ command status.	Access token và lệnh bật.	Cloud tạo lệnh, Gateway thực thi, đèn bật, App hiển thị trạng thái kết thúc.	TODO: Chạy trên hệ thống đầy đủ.	TODO: Chưa đánh giá.	TODO: Ảnh App, response API, MQTT trace và ảnh đèn.
HW-06	Luật tự động hóa	Kiểm tra event kích hoạt action.	Tạo luật công tắc hoặc chuyển động điều khiển đèn, sau đó phát trigger.	Rule hợp lệ và trigger.	Gateway thực hiện action; Cloud ghi automation event nếu có.	TODO: Chạy lại với thiết bị thật.	TODO: Chưa đánh giá.	TODO: Rule, log Gateway và trạng thái đèn.
HW-07	Mất kết nối và phục hồi	Kiểm tra lỗi mạng.	Ngắt kết nối Gateway-Broker, gửi lệnh, kết nối lại và quan sát trạng thái.	Mất mạng có kiểm soát.	Lệnh không bị báo executed giả; Gateway và thiết bị cập nhật lại trạng thái sau khi phục hồi.	TODO: Bổ sung timeline thực nghiệm.	TODO: Chưa đánh giá.	TODO: Log trước, trong và sau lỗi.

Mã	Tên	Tóm tắt	Thao tác chi tiết	Đầu vào	Kết quả mong đợi	Kết quả thực tế	Đánh giá	Evidence
SW-01	Xác thực và RBAC	Kiểm tra login, token, vai trò và phạm vi nhà.	Chạy nhóm test xác thực và phân quyền.	Bộ test hiện tại.	Tất cả test trong file đạt.	21 test đạt, không có test lỗi.	Đạt	Output 21 passed, ngày 12/06/2026.
SW-02	Thiết bị, lệnh và MQTT	Kiểm tra API, handler, trạng thái online và event Gateway.	Chạy năm file test commands, devices, MQTT và lifecycle.	Bộ test hiện tại.	Tất cả test được chọn đạt.	27 test đạt, không có test lỗi.	Đạt	Output 27 passed, ngày 12/06/2026.
SW-03	Tự động hóa và provisioning	Kiểm tra luật, event và phiên thêm thiết bị.	Chạy bốn file test automation và provisioning.	Bộ test hiện tại.	Tất cả test được chọn đạt.	61 test đạt, không có test lỗi.	Đạt	Output 61 passed, ngày 12/06/2026.
SW-04	Mobile App	Kiểm tra model, repository, view model và widget.	Chạy flutter test trong mobile_app/.	Bộ test Flutter hiện tại.	Toàn bộ test đạt, không có widget overflow trong test màn hình hẹp.	104 test đạt.	Đạt	Output A11 tests passed, ngày 12/06/2026.
SW-05	Toàn bộ Cloud suite	Kiểm tra toàn bộ test Cloud.	Chạy pytest cloud/tests -q.	Bộ test Cloud hiện tại.	Không có test lỗi.	177 đạt, 5 lỗi và 21 bỏ qua. Năm lỗi do thiếu tài liệu production được contract test yêu cầu.	Chưa đạt	Output pytest và danh sách năm tệp thiếu, ngày 12/06/2026.

Các test SW-01 đến SW-04 chứng minh các nhóm chức năng phần mềm tương ứng đang đạt trong phạm vi test đã chọn. Kết quả này không đủ để đổi trạng thái các test HW-01 đến HW-07. Những test phần cứng vẫn cần evidence từ hệ thống đang chạy.

5.4 Phân tích kết quả phần mềm

5.4.1 Cloud Backend

Nhóm xác thực và Role-Based Access Control (RBAC), tức phân quyền theo vai trò, có 21 test đạt. Nhóm thiết bị, lệnh, MQTT và vòng đời online có 27 test đạt. Nhóm tự động hóa và thêm thiết bị có 61 test đạt. Các kết quả này cho thấy những đường xử lý trọng tâm có regression test và đang chạy đúng trong môi trường test.

Khi chạy toàn bộ `cloud/tests`, kết quả là 177 test đạt, 5 test lỗi và 21 test bỏ qua. Năm lỗi đều là `FileNotFoundException` tại các contract test yêu cầu những tệp sau:

- `docs/production/production-operations.md`;
- `docs/production/production-acceptance-checklist.md`;
- `docs/production/production-release.md`;
- `docs/production/release-notes-v0.9.0-rc.1.md`.

Một tệp bị yêu cầu bởi hai test nên tổng số test lỗi là năm. Đây là lỗi thiếu artifact tài liệu production, không phải bằng chứng cho thấy 177 test còn lại sai. Tuy nhiên, suite tổng vẫn được đánh giá *Chưa đạt* cho tới khi các contract test này được xử lý.

5.4.2 Mobile App

Lệnh `flutter test` chạy 104 test và kết thúc với thông báo `All tests passed`. Bộ test bao phủ model, repository, view model, màn hình đăng nhập, dashboard và trường hợp viewport hẹp. Kết quả này chứng minh code Flutter đạt test tự động hiện có; nó không thay thế kiểm thử thủ công trên điện thoại thật hoặc kiểm thử với Cloud/Gateway thật.

5.5 Evidence cần bổ sung cho kiểm thử phần cứng

Mỗi test phần cứng cần một bộ evidence đủ để truy lại toàn bộ luồng. Với test join, cần log EUI-64 và registry. Với test đèn, cần command request, command reply, reported state và ảnh trạng thái vật lý. Với công tắc hoặc cảm biến, cần log sự kiện, API response và hành vi thiết bị đích nếu có automation.

Ảnh giao diện đơn lẻ chỉ chứng minh trạng thái hiển thị tại một thời điểm. Để chứng minh lệnh xuyên suốt, cần ghép ảnh hoặc video với command ID, MQTT trace và trạng thái lưu trong Cloud. Nếu evidence không chứa định danh hoặc thời gian, không thể chắc chắn các ảnh thuộc cùng một lần chạy.

5.6 Hạn chế và phạm vi mở rộng

Bộ test hiện tại có coverage tốt hơn ở Cloud và Mobile App so với phần cứng. Các tình huống mất điện, nhiễu sóng, nhiều thiết bị join đồng thời và vận hành dài ngày chưa có evidence trong báo cáo. Đây là các test cần ưu tiên trước khi đánh giá độ ổn định hoặc độ trễ của hệ thống.

Cập nhật firmware qua mạng, còn gọi là Over-the-Air (OTA), không thuộc phạm vi triển khai và kiểm thử của phiên bản hiện tại. Nội dung này chỉ được giữ như một khả năng mở rộng sau khi các luồng join, điều khiển, báo trạng thái, phân quyền và phục hồi kết nối đã có evidence đầy đủ.

5.7 Tổng kết chương

Chương 5 đã tách rõ kết quả phần mềm có thể lặp lại bằng lệnh test và các test phần cứng còn thiếu evidence. Bốn nhóm test trọng tâm đạt, trong khi Cloud suite tổng chưa đạt do năm contract test thiếu tài liệu production. Các test thiết bị và test xuyên suốt chưa được ghi thành công cho tới khi có log, API response, ảnh hoặc video tương ứng.

CHƯƠNG 6

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Chương này tổng kết các kết quả có thể kiểm chứng từ mã nguồn và output kiểm thử, đồng thời nêu những phần còn thiếu evidence. Các hướng phát triển tập trung vào độ ổn định, khả năng quan sát, bảo mật và trải nghiệm vận hành của hệ thống.

6.1 Kết luận chung

Đồ án đã xây dựng được cấu trúc phần mềm cho một hệ thống Smart Building gồm thiết bị Zigbee, Gateway viết bằng C, Message Queuing Telemetry Transport (MQTT) Broker, Cloud Backend FastAPI, cơ sở dữ liệu PostgreSQL và Mobile App Flutter. Các lớp trao đổi qua hợp đồng dữ liệu rõ ràng thay vì để ứng dụng truy cập trực tiếp mạng Zigbee.

Ở lớp thiết bị, repository có firmware cho EFR32 Network Co-Processor (NCP), đèn, công tắc và cảm biến chuyển động. Ở lớp Gateway, Z3GatewayHost dùng libmosquitto để nhận lệnh và phát trạng thái, sự kiện, registry hoặc phản hồi. Ở lớp Cloud, hệ thống có API, mô hình dữ liệu, quản lý vòng đời lệnh, xác thực, phân quyền theo vai trò và kiểm soát phạm vi nhà/thiết bị. Mobile App có đăng nhập, lưu phiên bảo mật, giám sát thiết bị, điều khiển đèn, tự động hóa và thêm thiết bị.

Kết quả test tự động ngày 12/06/2026 cho thấy nhóm xác thực/phân quyền có 21 test đạt, nhóm thiết bị/lệnh/MQTT có 27 test đạt, nhóm tự động hóa/thêm thiết bị có 61 test đạt và Mobile App có 104 test đạt. Toàn bộ Cloud suite chưa xanh vì 5 contract test thiếu tài liệu production; kết quả tổng là 177 đạt, 5 lỗi và 21 bỏ qua.

Báo cáo chưa kết luận các test phần cứng và luồng xuyên suốt đã đạt nếu chưa có log, ảnh, API response hoặc video tương ứng. Cách ghi nhận này giữ ranh giới rõ giữa chức năng có trong source, chức năng đã được test tự động và hành vi đã được xác nhận trên thiết bị thật.

6.2 Kiến thức và kỹ năng đạt được

Quá trình thực hiện giúp nhóm hiểu cách Zigbee tổ chức Coordinator, Router, End Device, cluster, attribute và command. Nhóm cũng làm việc với giao tiếp Host-NCP qua EmberZNet Serial Protocol (EZSP), cơ chế publish/subscribe của MQTT và vòng đời bất đồng bộ của lệnh điều khiển.

Ở phần mềm phía trên, nhóm thực hành xây dựng API bằng FastAPI, ánh xạ dữ liệu quan hệ bằng SQLAlchemy, quản lý phiên đăng nhập, phân quyền theo vai trò và lưu token trên ứng dụng di động. Việc tách repository, view model và giao diện trong Flutter giúp kiểm thử từng phần mà không phụ thuộc thiết bị thật.

Nhóm sử dụng Jira để quản lý đầu việc, Git để quản lý lịch sử thay đổi và GitHub để chia sẻ repository. Quá trình tích hợp cũng rèn luyện cách đọc log, thiết kế test case, phân biệt lỗi phần mềm với lỗi môi trường và chỉ kết luận khi evidence đủ mạnh.

6.3 Hướng phát triển tiếp theo

6.3.1 Hoàn thiện phần cứng và kiểm thử dài hạn

Cần chạy lại toàn bộ test join, điều khiển đèn, sự kiện công tắc, cảm biến chuyển động và tự động hóa trên thiết bị thật. Mỗi lần chạy cần lưu command ID, log Gateway, MQTT trace, API response và ảnh hoặc video. Sau khi luồng cơ bản ổn định, hệ thống nên được kiểm thử nhiều node, nhiều sóng, mất điện và vận hành liên tục.

Cảm biến chuyển động có thể được mở rộng bằng cảm biến PIR hoặc radar hiện diện thay cho nguồn kích hoạt mô phỏng. Firmware cần xử lý chống dội và lọc nhiễu trước khi phát thay đổi thuộc tính.

Bổ sung node chấp hành: Hệ thống nên thêm các actuator, tức node chấp hành biến lệnh phần mềm thành tác động vật lý. Các node ưu tiên gồm ổ cắm thông minh, relay nhiều kênh, dimmer điều chỉnh độ sáng, động cơ rèm và bộ điều khiển điều hòa. Mỗi loại cần có device type, cluster Zigbee, trạng thái và tập lệnh riêng; không nên dùng hợp đồng của đèn để mô phỏng mọi actuator.

Cập nhật firmware từ xa: Over-the-Air (OTA) là cơ chế phân phối firmware qua mạng thay vì nạp trực tiếp bằng cáp. Hướng phát triển này cần bootloader phù hợp trên thiết bị, image có phiên bản và chữ ký, server/client của Zigbee OTA Cluster, cơ chế tải và kiểm tra image tại Gateway, API quản lý chiến dịch tại Cloud và khả năng phục hồi khi cập nhật thất bại [10]. OTA chỉ nên triển khai sau khi các luồng điều khiển và phục hồi kết nối hiện tại có evidence ổn định.

6.3.2 Nâng cấp Gateway

Gateway nên được chuyển sang máy tính nhúng chuyên dụng và chạy như một dịch vụ tự khởi động. Hàng đợi lệnh, registry và rule engine cần có metric để theo dõi số bản tin, lỗi phân loại, độ đầy hàng đợi và thời gian xử lý.

Rule engine cục bộ cần hỗ trợ đồng bộ phiên bản luật, rollback khi cấu hình không hợp lệ và hành vi xác định khi mất kết nối Cloud. Các test phục hồi phải chứng minh Gateway không thực thi lặp một lệnh sau khi reconnect.

6.3.3 Nâng cấp Cloud và Mobile App

Cloud cần hoàn thiện các tài liệu production mà contract test đang yêu cầu, gồm vận hành, acceptance checklist, release runbook và release notes. Hệ thống cũng cần backup/restore cơ sở dữ liệu, giám sát tập trung và cảnh báo khi Gateway hoặc thiết bị ngoại tuyến.

Mobile App hiện truy vấn lại trạng thái lệnh. Có thể bổ sung WebSocket hoặc Server-Sent Events (SSE), tức cơ chế máy chủ chủ động đẩy sự kiện, để giảm độ trễ hiển thị. Thay đổi này cần giữ REST API làm nguồn dữ liệu bền vững và xử lý trường hợp App mất kết nối.

Mở rộng mô hình phân quyền: Ba vai trò hiện tại là admin, parent và viewer. Khi áp dụng cho tòa nhà lớn hơn, hệ thống có thể bổ sung building_manager cho người quản lý tòa nhà, technician cho kỹ thuật viên bảo trì, tenant cho người thuê và guest cho khách tạm thời. Mỗi vai trò cần ma trận quyền, phạm vi theo site/tầng/phòng, thời hạn truy cập và audit log; không chỉ bổ sung tên role trong cơ sở dữ liệu.

6.3.4 Tăng cường bảo mật

Cấu hình Transport Layer Security (TLS) và mutual TLS (mTLS), tức xác thực chứng thư ở cả hai phía, cần được kiểm tra trên môi trường triển khai thực tế. MQTT Access Control List

(ACL) phải giới hạn từng client trong đúng tenant, site và gateway.

Cloud cần bổ sung chính sách mật khẩu, thu hồi phiên, xoay vòng khóa và audit log cho thao tác nhạy cảm. Kiểm thử bảo mật phải bao gồm người dùng khác nhà, token hết hạn, refresh token đã thu hồi và client MQTT cố truy cập topic ngoài namespace.

6.4 Kết luận chương

Hệ thống đã có nền tảng kỹ thuật đủ để tiếp tục kiểm thử và hoàn thiện theo hướng sản phẩm. Ưu tiên tiếp theo không phải mở rộng thêm nhiều tính năng, mà là hoàn tất evidence phần cứng, làm xanh toàn bộ test suite, bổ sung tài liệu vận hành và đo độ ổn định trong điều kiện thực tế.

TÀI LIỆU THAM KHẢO

- [1] Connectivity Standards Alliance. (2023). *Zigbee Specification*. Revision 23. Zigbee Document 05-3474-23.
- [2] Zigbee Alliance. (2019). *Zigbee Cluster Library Specification*. Revision 8. Zigbee Document 07-5123.
- [3] Silicon Labs. (n.d.). *UG103.2: Zigbee Fundamentals*. Deprecated version.
- [4] HiveMQ. (n.d.). *MQTT Essentials: The Ultimate Guide to the MQTT Protocol for IoT Messaging*. Version 2.0.
- [5] EMQ Technologies Inc. (2024). *Mastering MQTT: Your Ultimate Tutorial for MQTT*. Release: R24-3.
- [6] Peña, E., & Legaspi, M. G. (2020). *UART: A Hardware Communication Protocol - Understanding Universal Asynchronous Receiver/Transmitter*. Analog Devices.
- [7] Silicon Labs. (n.d.). *UG100: EZSP Reference Guide*. EmberZNet Serial Protocol.
- [8] Silicon Labs. (n.d.). *AN1278: Z3Gateway Example Application*. Zigbee NCP-host gateway using EZSP.
- [9] Google. (n.d.). *Flutter Documentation*. <https://docs.flutter.dev>.
- [10] Silicon Labs. (n.d.). *AN728: Over-the-Air Bootload Server and Client Setup*. Zigbee OTA Upgrade Cluster.